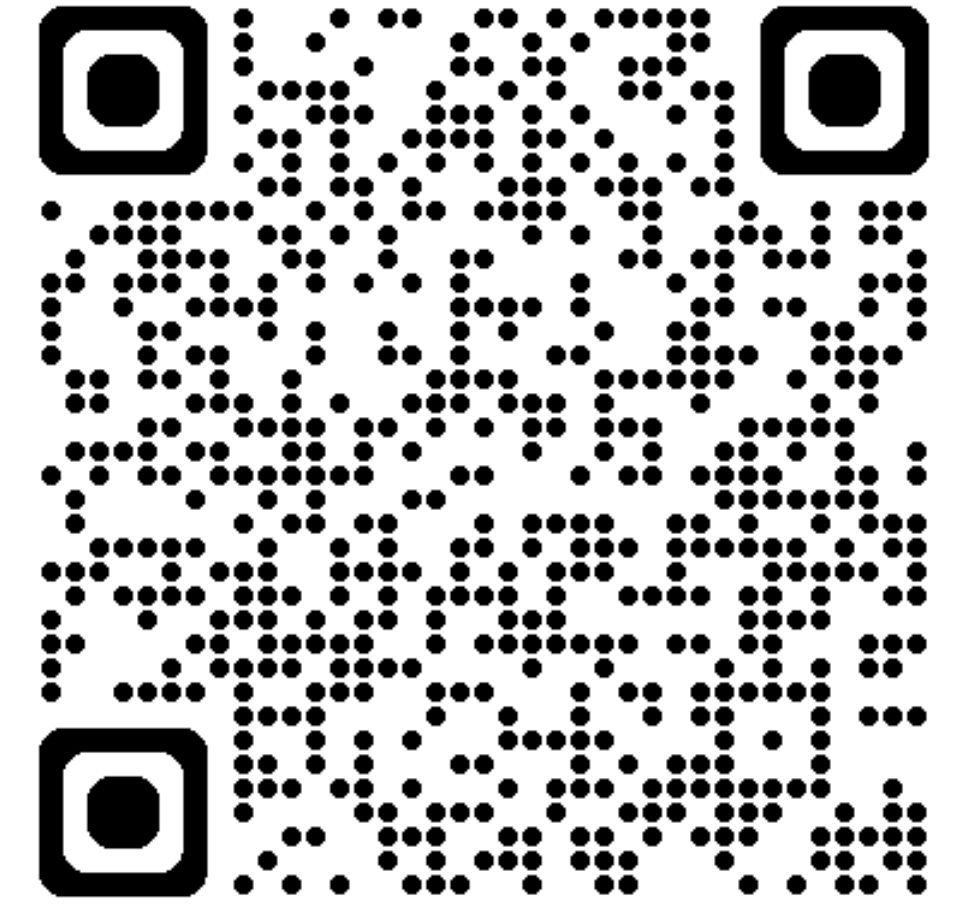


Übungsstunde 6

Heute:

- Rückblick & Repetitions Quiz
- BubbleSort
- InsertionSort
- Laufzeiten
- Ausblick Code Expert: EX5



Vielen Dank für das Feedback!

- ich werde darauf achten künftig **mehr Interaktivität und Aufgaben zur Anwendung der Theorie** zu implementieren!
- Auch bzgl. des Tempos versuche ich mich im Griff zu haben.

Verlauf des Semesters

Informatik nimmt diese Woche Fahrt auf!

wer bisher Mühe hatte: Die Mühe mit Python wird sich mit der Zeit legen, es wird künftig viel mehr darum gehen Wege zu finden etwas zu programmieren als rein die Syntax zu kennen

...Letzte Woche

Abschluss Python basics

- **Klassen**
 - **Methoden**
 - **Vererbung**
- **Lambda Funktionsausdrücke**

Ich war nicht da. Teilt mir gerne mit, wenn ich noch ein Thema dazu vertiefen soll.

Warm-up: Repetitionsquiz

Inwiefern werden sich **x** und **y** unterscheiden?

```
import random
#random.randint(n) generiert eine zufällige zahl [a,b]

x = [random.randint(0,50)] * 10
y = [random.randint(0,50) for _ in range(10)]
```

Warm-up: Repetitionsquiz

Was ist jeweils das Ergebnis dieser Ausdrücke/Zellen?

```
(lambda x: x[::-1].upper())("exam")
```

```
dbl= (lambda x: return x*2)
```

```
print(dbl(10))
```

```
import numpy as np
```

```
arr = np.arange(6).reshape(2, 3)
```

```
# arr sieht so aus:
```

```
# [[0, 1, 2],
```

```
#  [3, 4, 5]]
```

```
res = arr[:, 1].sum()
```

```
print(res)
```


Eine weitere Aufgabe

Inspiziert von MAVT August 2025. (Selbst erfundene Aufgabe)

Vervollständigen Sie die Funktion, welche einen String `text` und einen weiteren String `char` als Eingabe nimmt. $\text{len}(\text{char}) = 1$ ist garantiert.

Ziel ist es, den Buchstaben `char` im String `text` zu vervielfachen.
Das Muster der Vervielfachung ist wie folgt definiert:

Eingabe:	<code>text, x</code>	$\Rightarrow$	<code>texxt</code>
	<code>text, t</code>	$\Rightarrow$	<code>ttexttt</code>
	<code>Text, t</code>	$\Rightarrow$	<code>textt</code>
	<code>seses, s</code>	$\Rightarrow$	<code>ssesssesssss</code>

```
def multiply_char(text, char):  
    #TODO: implementieren Sie die Funktion  
  
    return '' #ändern Sie diesen Wert !  
  
# Test mit den Beispielen:  
print(multiply_char("text", "x")) # Output: texxt  
print(multiply_char("text", "t")) # Output: ttexttt  
print(multiply_char("Text", "t")) # Output: textt  
print(multiply_char("seses", "s")) # Output: ssesssesssss
```

top, beginnen wir: BubbleSort

„Wozu muss ich diesen random Algorithmus kennen?“

=> er wird an der **Prüfung** als bekannt vorausgesetzt.

bsp Aug. 2025 MAVT, Vergleich eines gegebenen Algorithmus mit BubbleSort 🔍

Bubblesort

start

5 2 8 4 1

Die originale liste. (gilt es zu sortieren)

5 > 2 8 4 1

- Vergleiche jedes Paar benachbarter Elemente der Reihe nach. Wenn das erste größer ist, tausche sie aus.

schritte

ende

Bubblesort

5 2 8 4 1

5 > 2 8 4 1

2 5 < 8 4 1



da die 5 grösser ist als die zwei werden sie „geswapped“. Nun vergleicht man die 5 mit der 8

Bubblesort

5 2 8 4 1

5 > 2 8 4 1

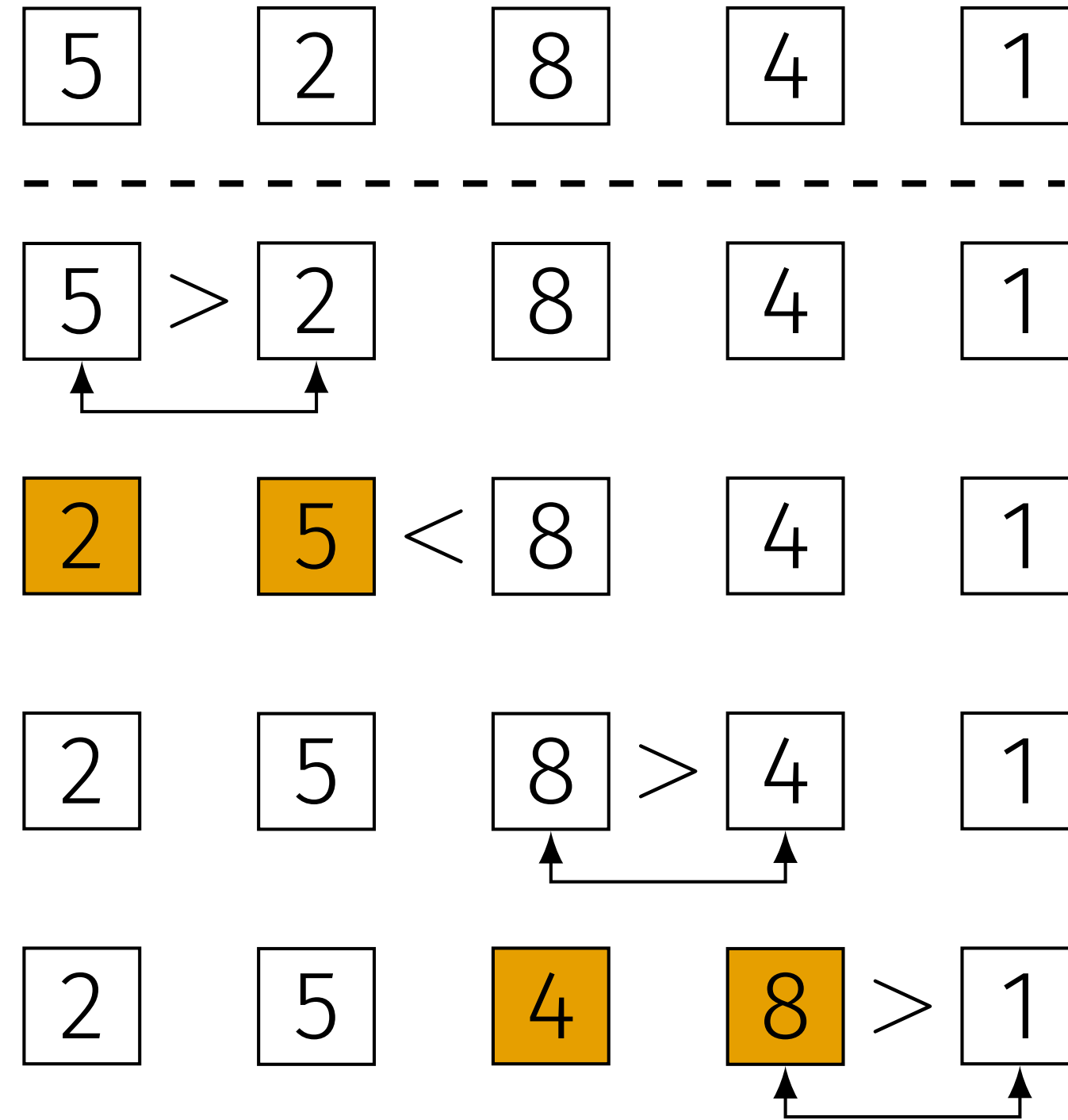
2 5 < 8 4 1

2 5 8 > 4 1

das grössere Element (8) ist bereits rechts von dem kleineren (5)

...deshalb gehen wir gleich weiter, und vergleichen nun die 8 mit der 4

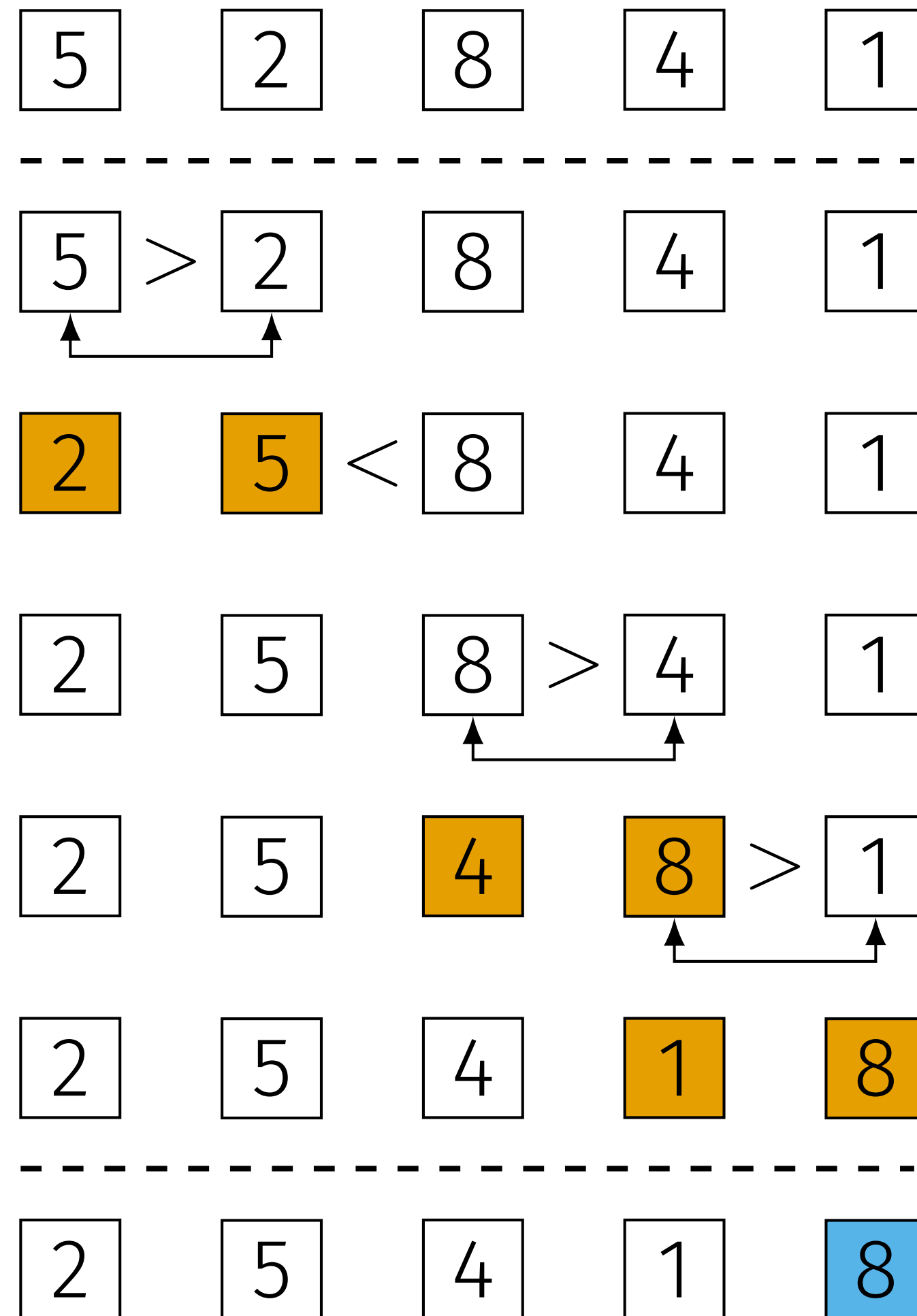
Bubblesort



erneut, wir swappen die 8 mit der 4.

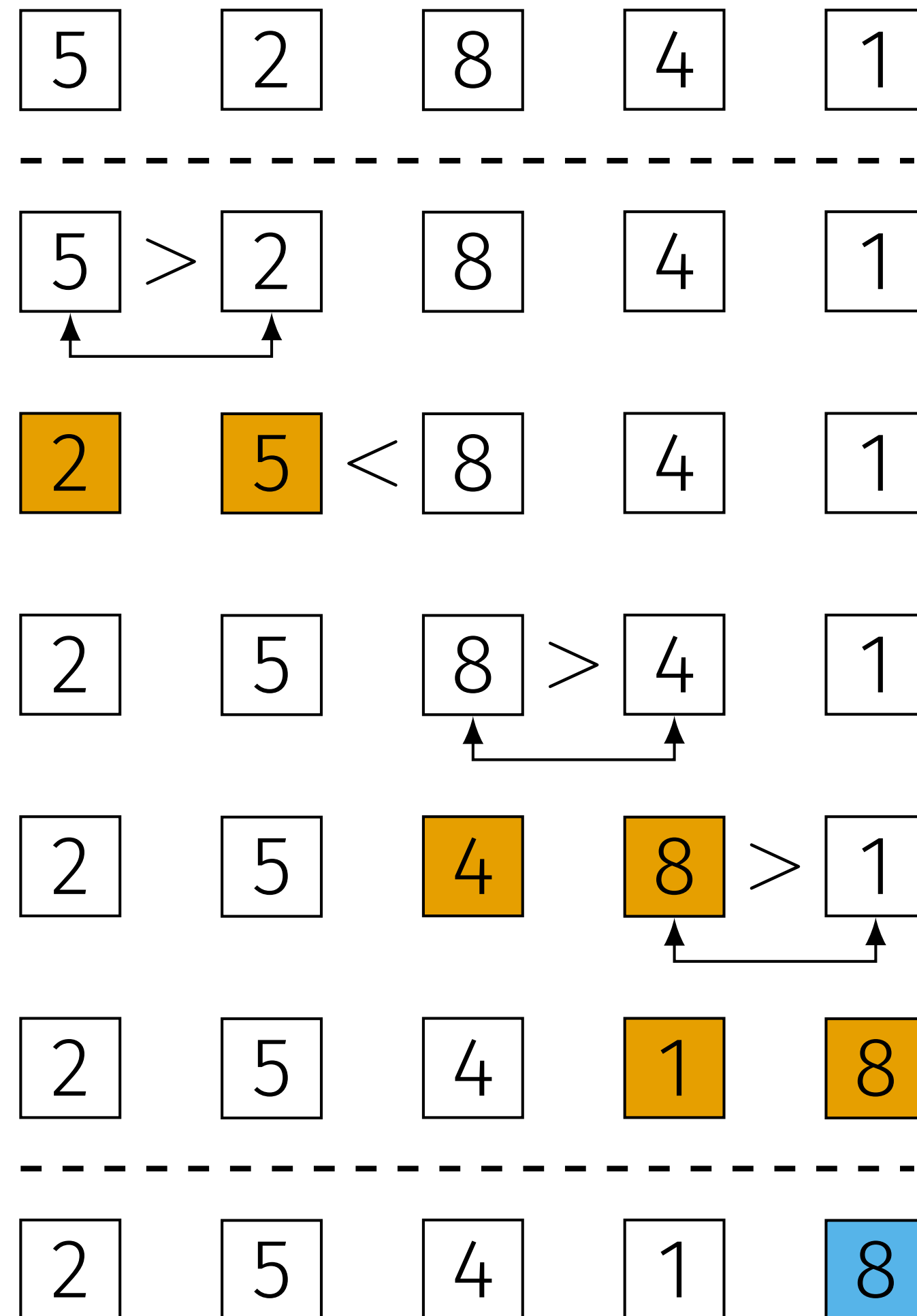
beachte: wir kümmern uns erst zu einem späteren Zeitpunkt um die position der 4

Bubblesort



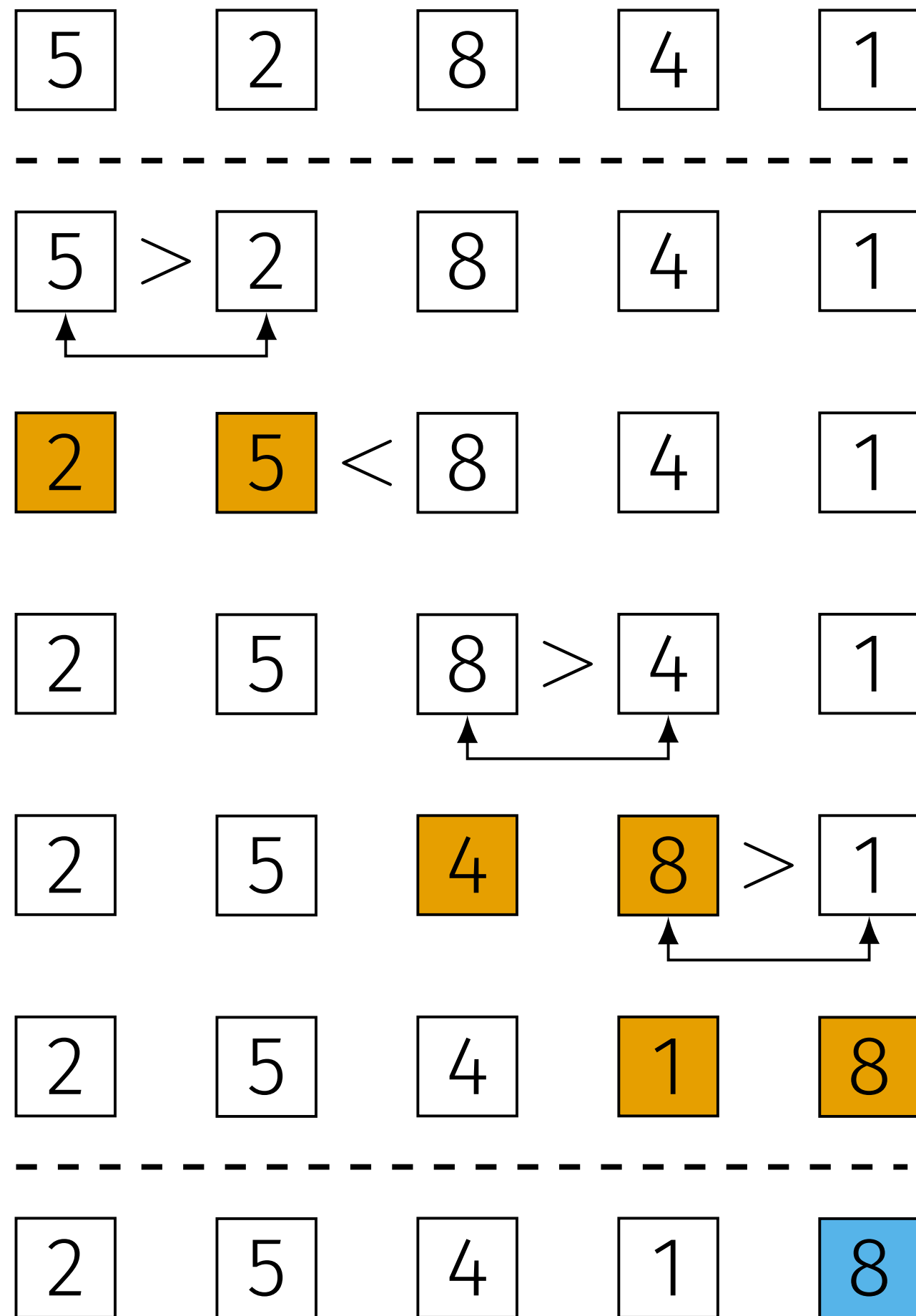
...und tauschen auch noch die 8 mit der 1.
nun ist die „**erste Iteration**“ abgeschlossen“

Bubblesort



Frage: In diesem Fall ist das grösste Element der Liste nach der ersten Iteration am Ende angelangt, **ist dies allgemein gültig?**

Bubblesort

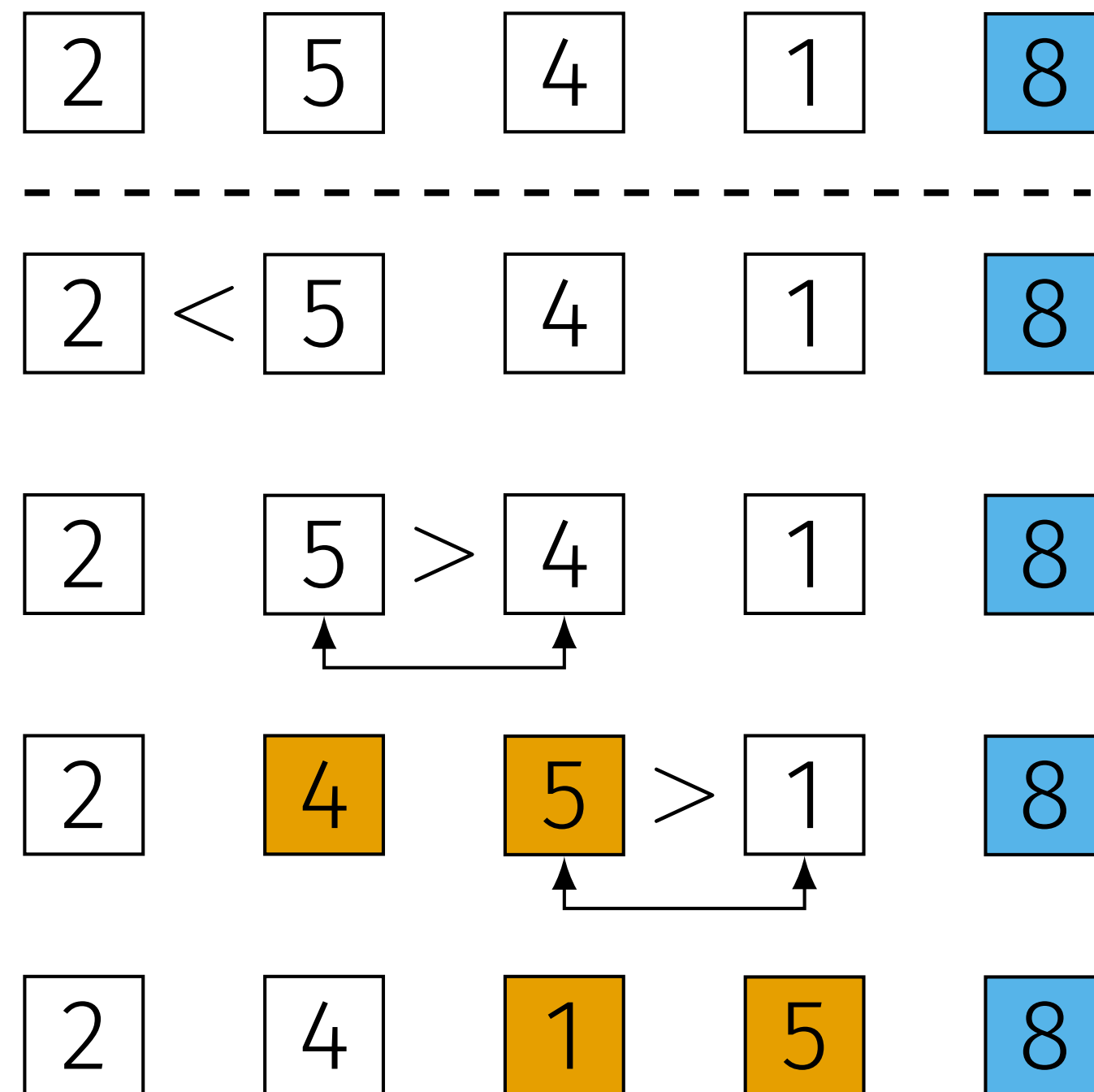


...Ja

- Vergleiche jedes Paar benachbarter Elemente der Reihe nach. Wenn das erste größer ist, tausche sie aus.
- Nach einer Iteration (Iteration 0) befindet sich das größte Element am Ende der Liste.

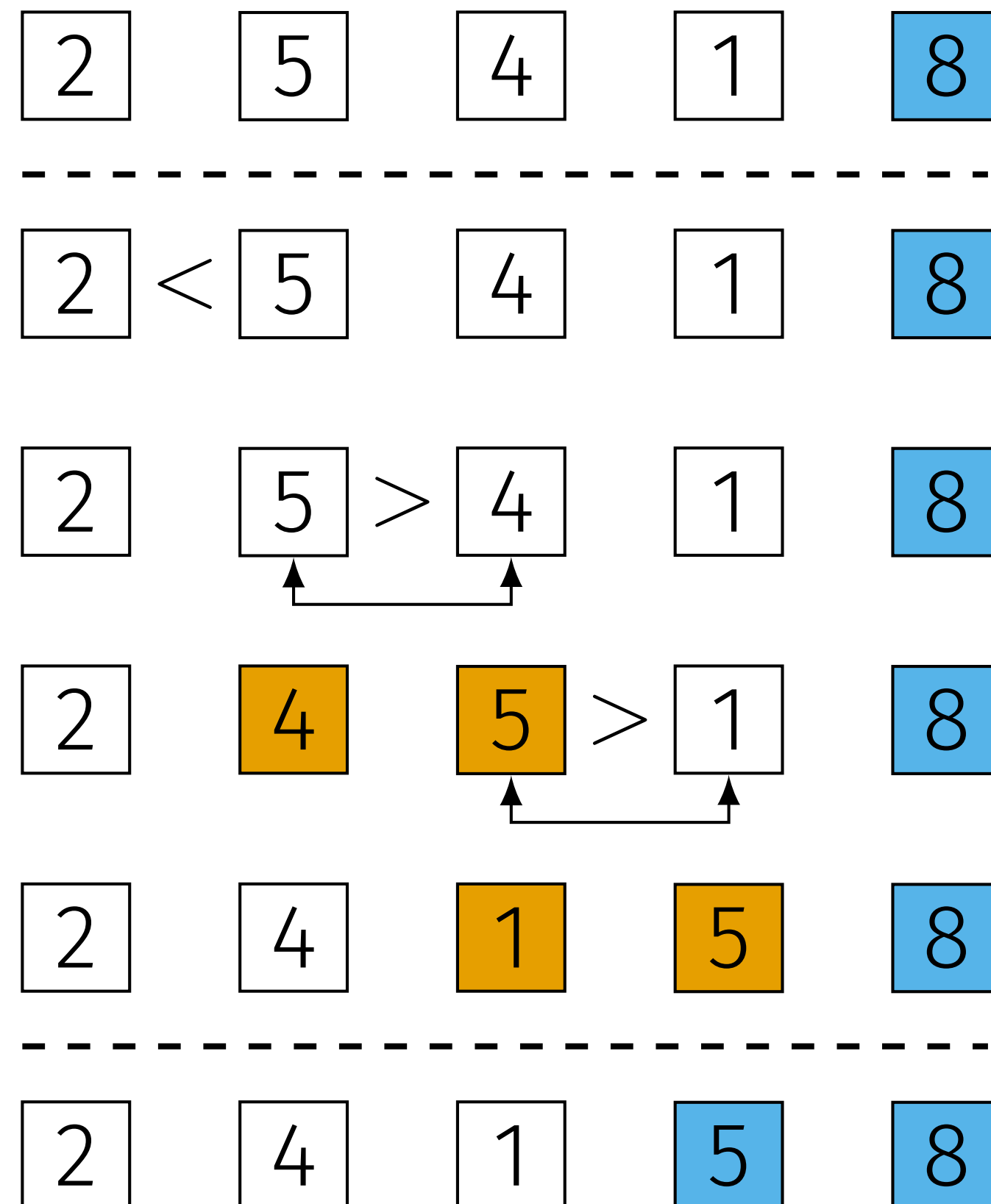
nach der ersten Iteration ist das grösste Element an seinem Platz angekommen. Nach der zweiter das zweitgrösste etc..

Bubblesort



- Schleife, bis die Liste sortiert ist.

Bubblesort



die **blauen Elemente sind gesetzt**, sprich sie befinden sich an ihrer finalen Position / „sind sortiert“

dies wird auch eine **(Schleifen)-Invariante** genannt.
Nach der Iteration i sind $li[-(i+1):]$ sortiert und bleiben unverändert.

Kurzer Verständnis-Check

True or False?

- Falls ich die Liste absteigend anstatt aufsteigend sortieren will, genügt es den Vergleichsoperator Umzukehren. (Sodass wir swappen, wenn das linke Element kleiner ist)
- Ein sauberer BubbleSort überprüft die invarianten Elemente, verschiebt sie aber nicht.
- Nachdem Bubble Sort zwei Elemente gewapped hat, sprechen wir von einer neuen Iteration.

Wie sieht das im Code aus?

Python Syntax, Elemente swappen

es geht ohne Zwischenvariable

```
li=[5,1,3,2,4]  
li[0], li[1]= li[1], li[0]  
li
```

✓ 0.0s

[1, 5, 3, 2, 4]

Wir können das Element i=1 dem index 0 zuweisen.

Und gleichzeitig das Element i=0 dem index 1.

Den Code konstruieren.

Was haben wir eigentlich vorhin gemacht?

- ➡ Wir sind von Links nach Rechts **gewandert**, und haben verglichen, ob das betrachtete Paar geordnet ist. Und ggF. das Paar getauscht.
- ➡ Wir wissen, dass nach dem ersten Durchlauf „**Wandern**“ das grösste Element final am Platz ist und das zweitgrösste nach dem zweiten Durchlauf.
- ➡ Es macht also Sinn, dieses „Wandern“ so oft zu machen, dass die Liste bis ganz nach links sortiert ist. Dies benötigt bei n Elementen $n-1$ **Wander-Durchgänge**.

Frage: Weshalb $n-1$?

Den Code konstruieren.

Was haben wir eigentlich vorhin gemacht?

=> Wir sind von Links nach Rechts **gewandert**, und haben verglichen, ob das betrachtete Paar geordnet ist. Und ggF. das Paar getauscht.

=> Wir wissen, dass nach dem ersten Durchlauf „**Wandern**“ das grösste Element final am Platz ist und das zweitgrösste nach dem zweiten Durchlauf.

=> Es macht also Sinn, dieses „Wandern“ so oft zu machen, dass die Liste bis ganz nach links sortiert ist. Dies benötigt bei n Elementen $n-1$ **Wander-Durchgänge**.

Frage: Weshalb $n-1$?

Wenn das zweit-kleinste Element seinen Platz gefunden hat, muss das kleinste zwangsläufig auch an seinem Platz sein. (Es gibt keine anderen freien Plätze)

„wandern“ coden.

wir iterieren durch die Liste um zu vergleichen & swappen.

wir gehen ganz normal durch die Liste durch und Vergleichen und swappen paarweise.

```
my_list = [3,4,2,1,5]
n=len(my_list)
#das ist "Wandern"
for j in range(n-1):
    #prüfen ob swap nötig ist. Beachte: Vergleich zwischen j und j+1!
    if my_list[j] > my_list[j+1]: # falls descending erwünscht: < verwenden.
        my_list[j], my_list[j+1] = my_list[j+1], my_list[j] # ggF. swappen
```

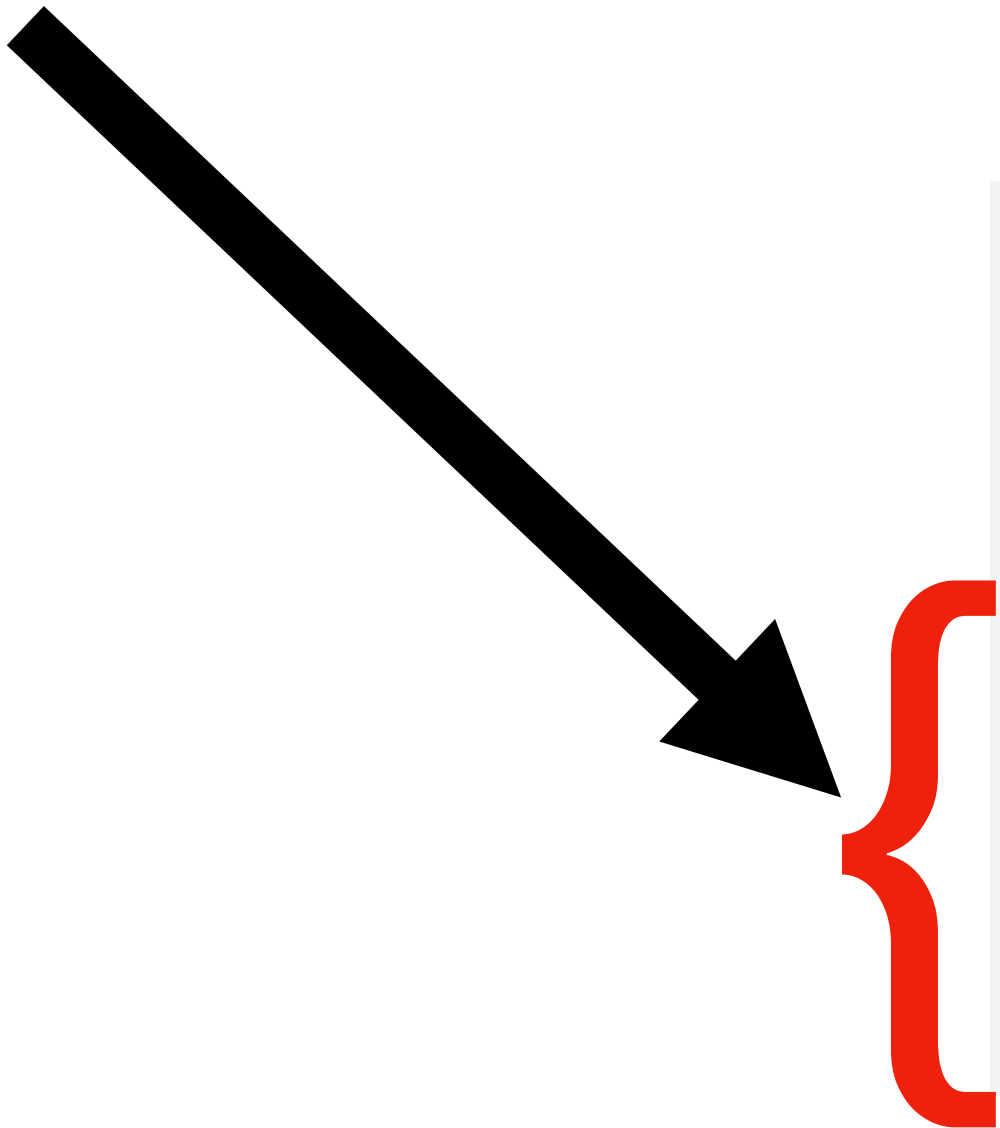
die Schleife geht aber nur bis $n-1$, denn wir „vergleichen nach vorne“ mit $j+1$

Vorhin haben wir gesehen, dass wir Wandern aber $n-1$ mal machen müssen, **wie können wir dies machen?**

... eine zweite Schleife

welche uns $n-1$ durchgänge „Wandern“ ermöglicht.

das wollen wir $n-1$ mal durchführen lassen



```
my_list = [3,4,2,1,5]
n=len(my_list)
#das ist "Wandern"
for j in range(n-1):
    #prüfen ob swap nötig ist. Beachte: Vergleich zwischen j und j+1!
    if my_list[j] > my_list[j+1]: # falls descending erwünscht: < verwenden.
        my_list[j], my_list[j+1] = my_list[j+1], my_list[j] # ggF. swappen
```

=> also packen wir **das** in eine zweite Schleife.

zweite Schleife implementieren

```
my_list = [3,4,2,1,5]
n=len(my_list)
for i in range(n-1):
    #das ist "Wandern"
    for j in range(n-1):
        #prüfen ob swap nötig ist. Beachte: Vergleich zwischen j und j+1!
        if my_list[j] > my_list[j+1]: # falls descending erwünscht: < verwenden.
            my_list[j], my_list[j+1] = my_list[j+1], my_list[j] # ggF. swappen
```

dieser code funktioniert. Er sortiert die Liste.

aber wir können ihn verbessern, wo?

Invarianten ausnutzen

wir müssen nicht in die **Invarianten** „reinwandern“

```
[4, 5, 6, 1, 100, 110, 250, 700]
```



die bereits **final sortierten** Elemente.



eigentlich wollen wir also nur diesen Teil durchwandern



wir wandern aber jedes mal diese Strecke ab

💡 wir haben ja aber bereits herausgefunden, dass unsere invarianten in $[-(i+1):]$ liegen. Dies können wir einfach implementieren.

Wandern begrenzen

„hiermit begrenzen wir nach und nach unseren Wanderbereich“



```
my_list = [3,4,2,1,5]
n=len(my_list)
for i in range(n-1):
    #das ist "Wandern"
    for j in range(n-1-i):
        #prüfen ob swap nötig ist. Beachte: Vergleich zwischen j und j+1!
        if my_list[j] > my_list[j+1]: # falls descending erwünscht: < verwenden.
            my_list[j], my_list[j+1] = my_list[j+1], my_list[j] # ggF. swappen
```

bemerkung: `[-(i+1):]=[-i-1:]`

BubbleSort

final, als Funktion definiert.

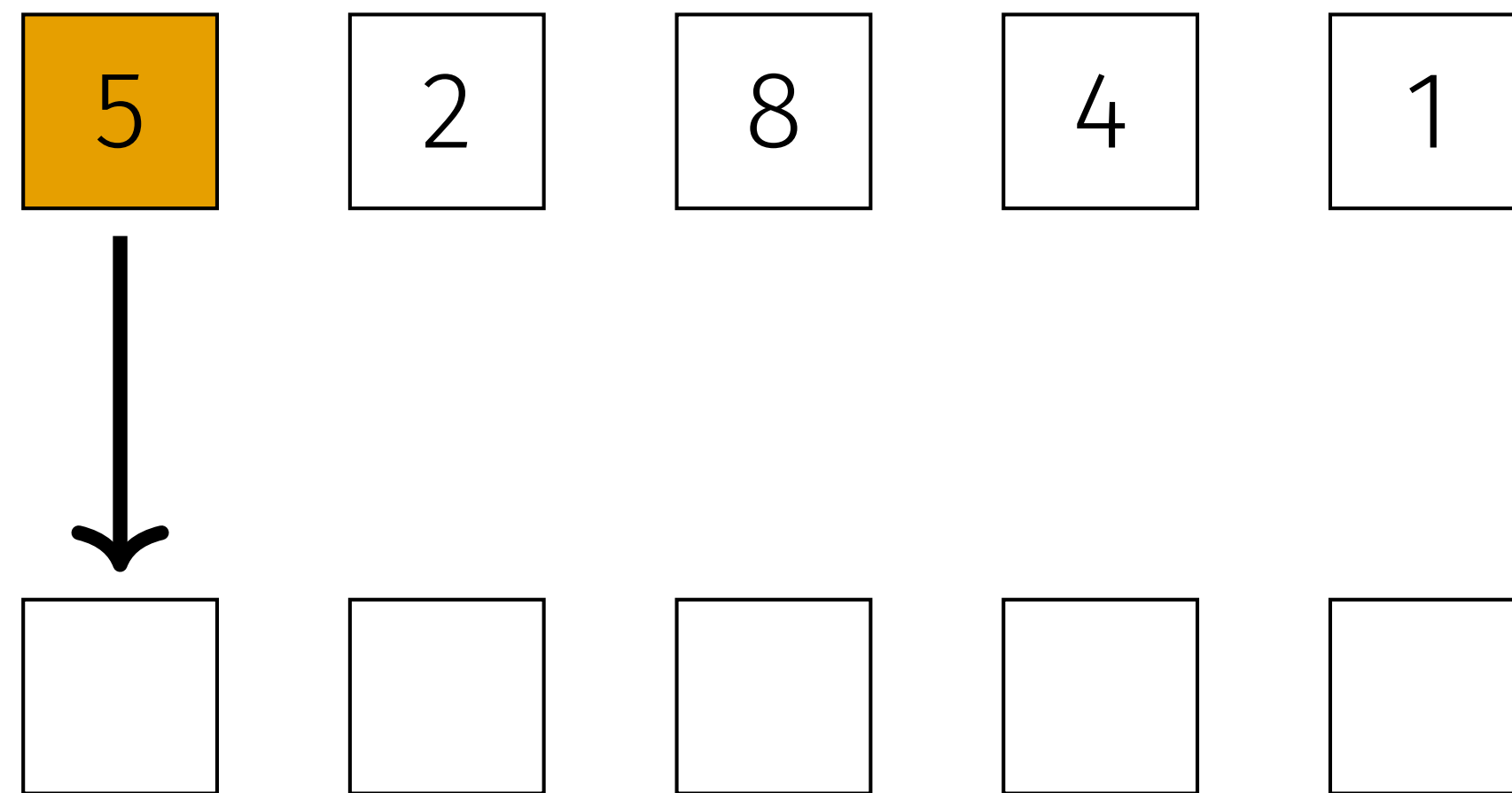
```
def bubbleSort(my_list):  
    n = len(my_list)  
  
    #äussere Schleife (wenn von Iteration die Rede ist, ist meist diese Schleife gemeint.)  
    #nach dem ersten durchlauf (i) ist das grösste Element am Ende, nach dem zweiten das zweitegrösste etc..  
    for i in range(n-1):  
        # Die innere Schleife, sie wird die Vergleiche und Swaps durchführen.  
        # am anfang von 0 bis zum zweit-letzten Element, von da schrittweise we  
        for j in range(n-i-1):  
            #prüfen ob swap nötig ist. Beachte: Vergleich zwischen j und j+1!  
            if my_list[j] > my_list[j+1]: # falls descending erwünscht: < verwenden.  
                my_list[j], my_list[j+1] = my_list[j+1], my_list[j] # ggF. swappen  
            #print(my_list) #optional hier ausgeben lassen für Verständnis.
```

.ipynb auf github.

1/3 von heute geschafft 

**weiter mit einem weiteren Algorithmus:
InsertionSort**

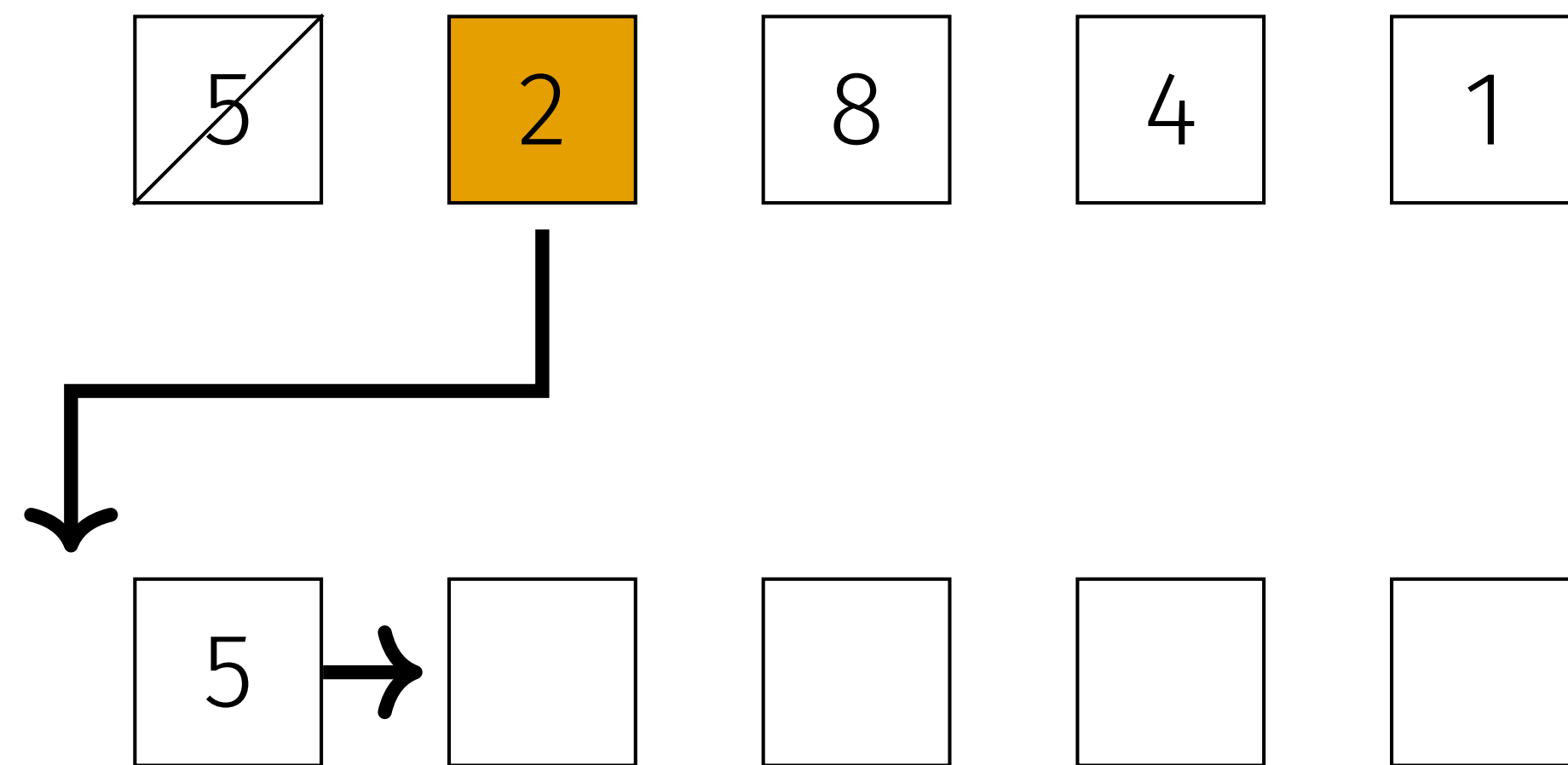
Insertion Sort: Konzept



- Mit einer zweiten Liste können wir einfach jedes Element an der korrekten Stelle einsortieren.

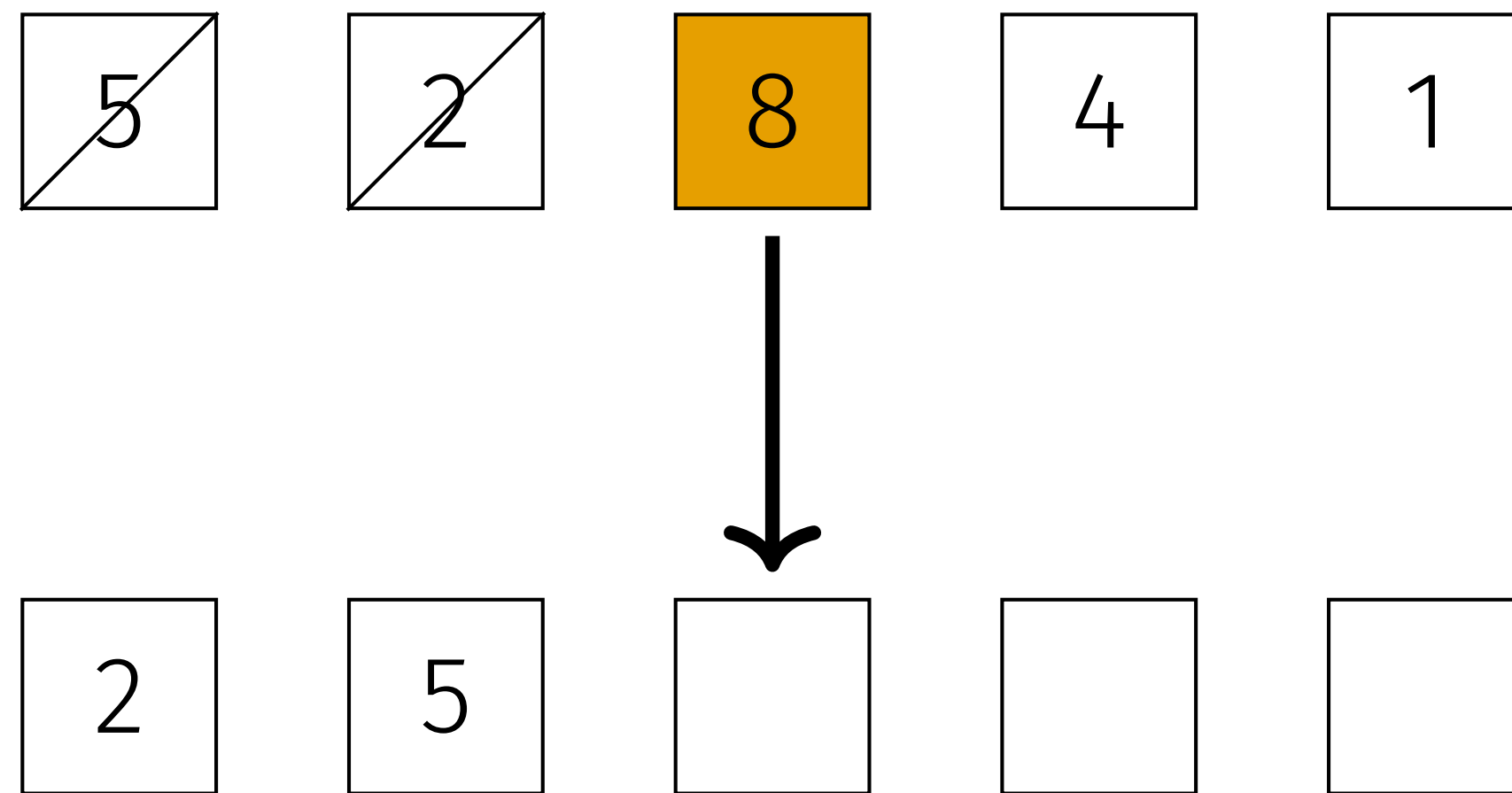
***ist nur die grundsätzliche Idee**

Insertion Sort: Konzept



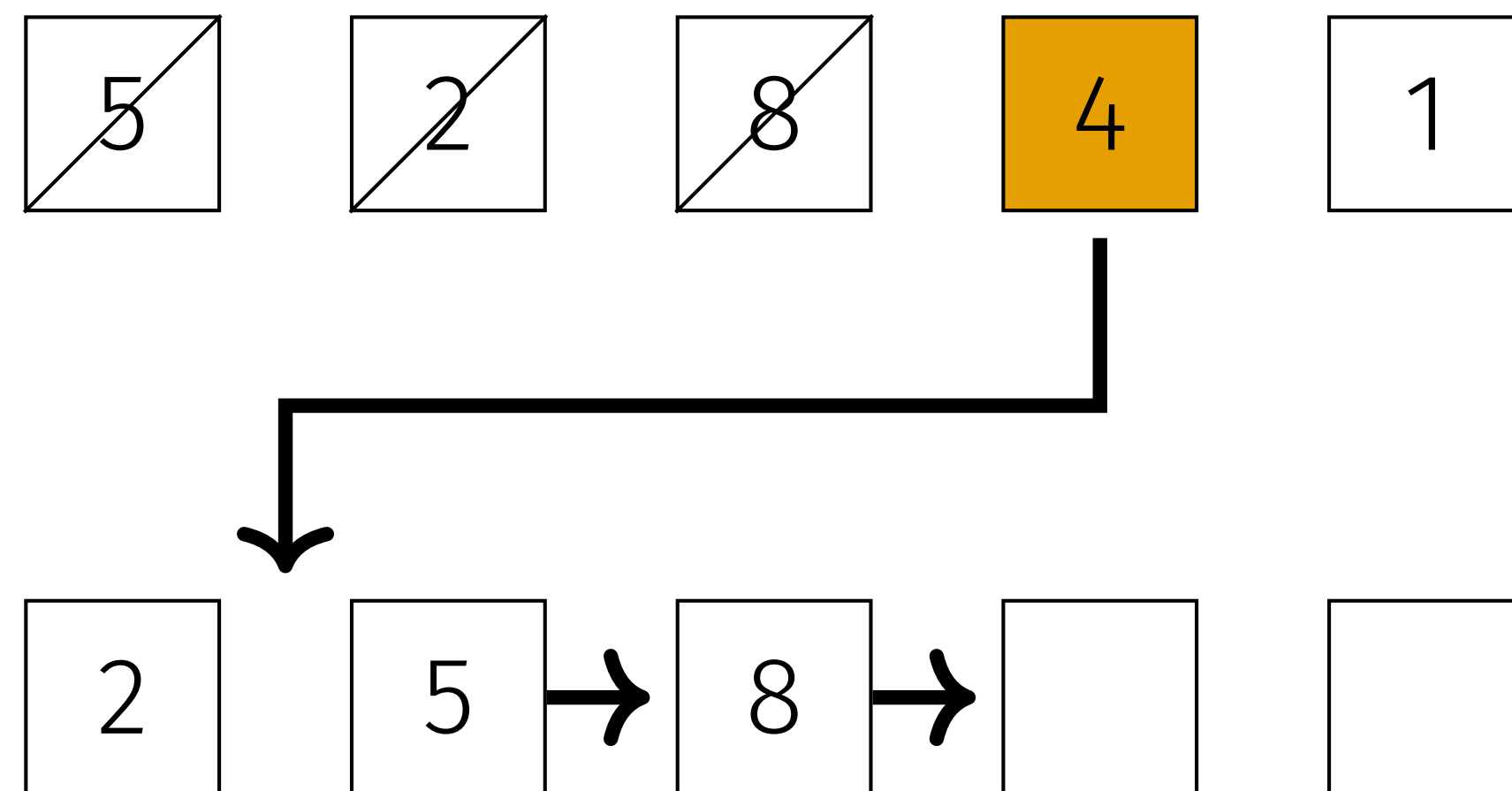
- Mit einer zweiten Liste können wir einfach jedes Element an der korrekten Stelle einsortieren.

Insertion Sort: Konzept



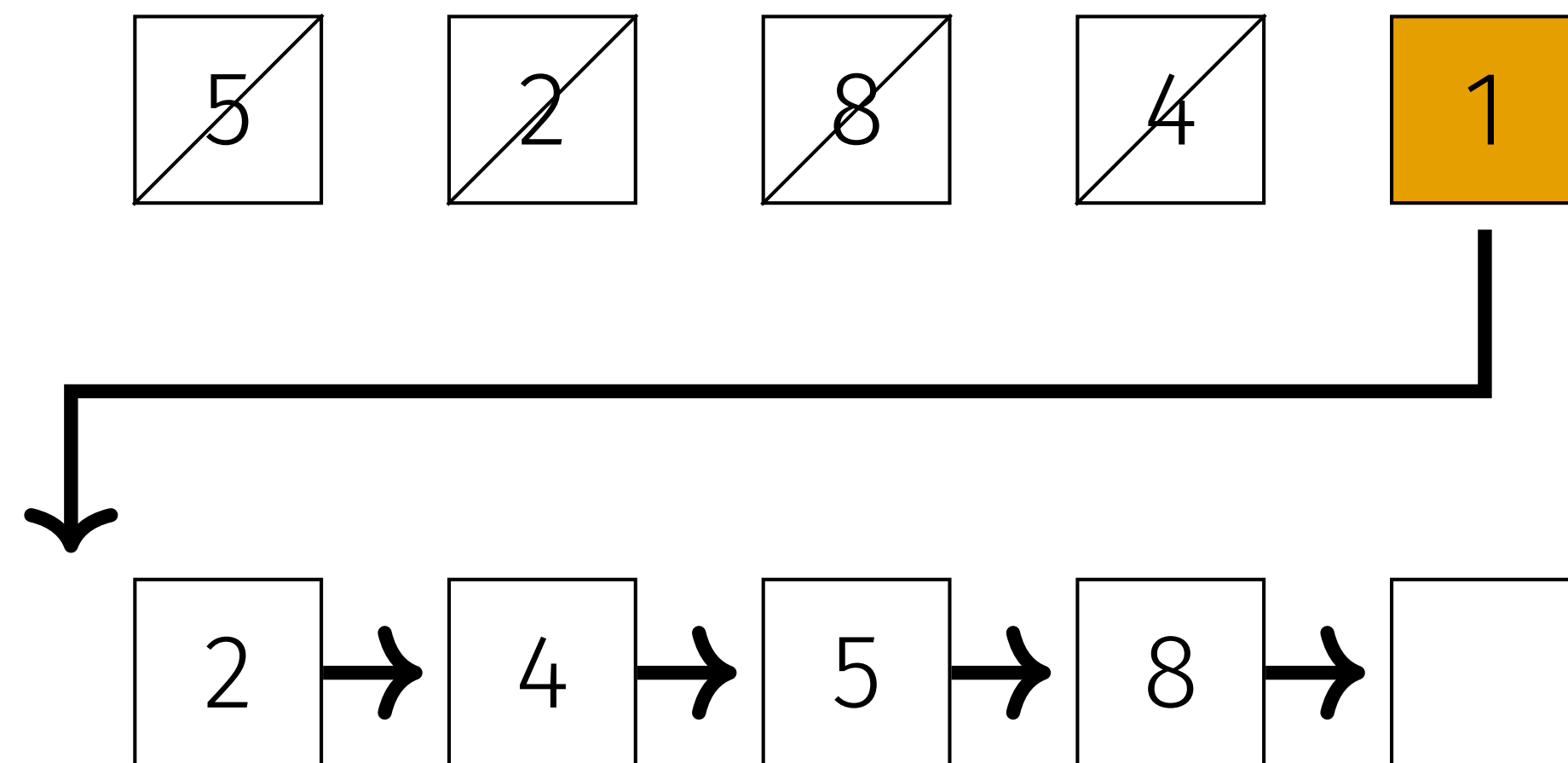
- Mit einer zweiten Liste können wir einfach jedes Element an der korrekten Stelle einsortieren.

Insertion Sort: Konzept



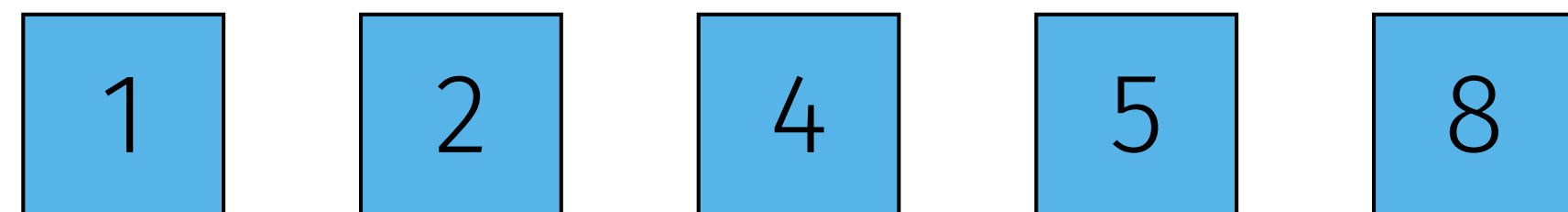
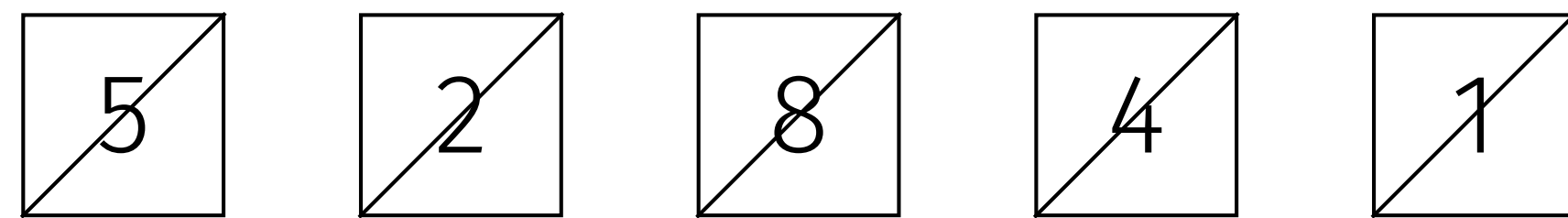
- Mit einer zweiten Liste können wir einfach jedes Element an der korrekten Stelle einsortieren.

Insertion Sort: Konzept



- Mit einer zweiten Liste können wir einfach jedes Element an der korrekten Stelle einsortieren.

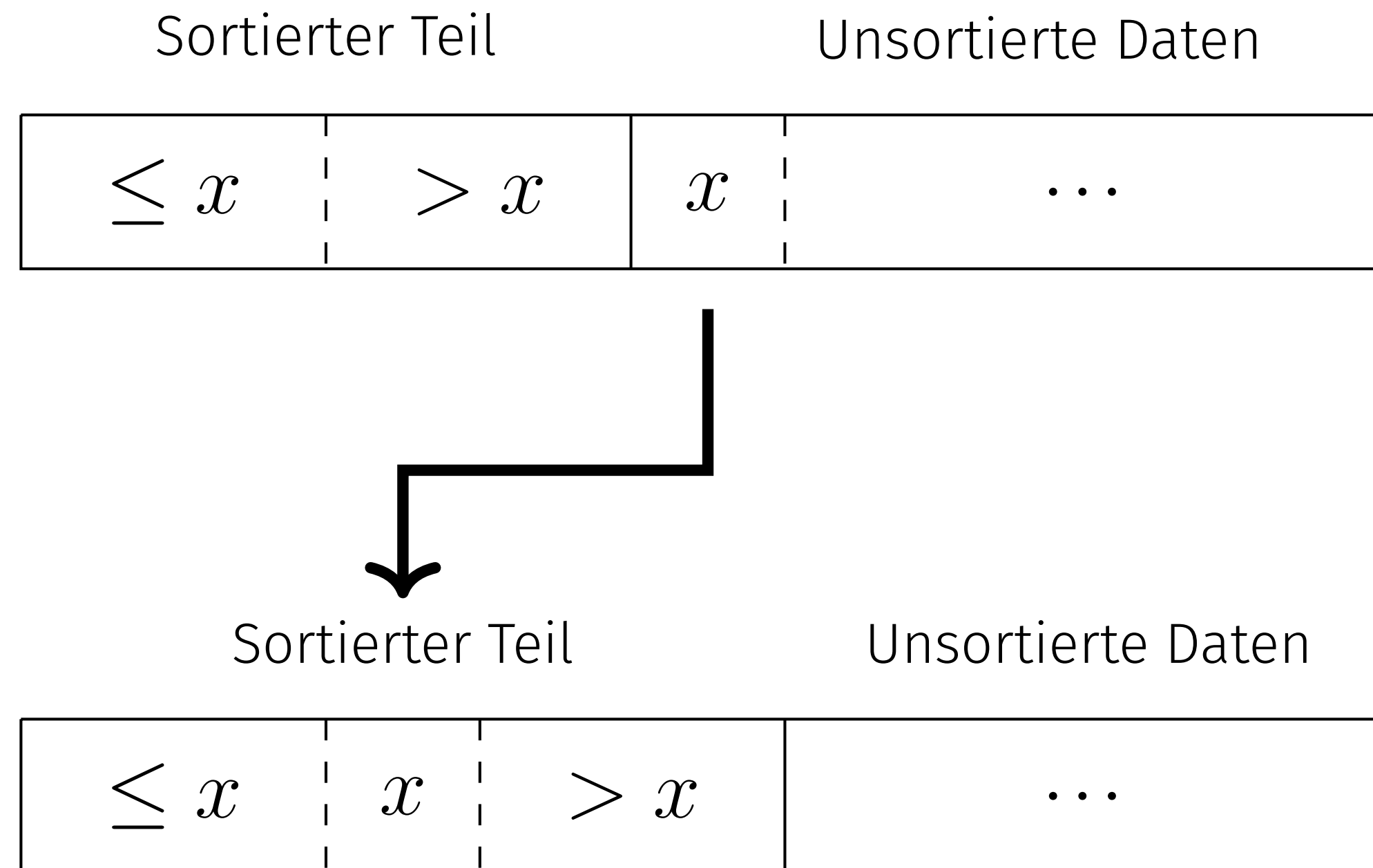
Insertion Sort: Konzept



- Mit einer zweiten Liste können wir einfach jedes Element an der korrekten Stelle einsortieren.
- **Problem:** Benötigt n zusätzlichen Speicherplatz.

weil wir eine neue Liste mit n Plätzen erstellen müssen!

Insertion Sort



Wir können den Speicherplatz, der in der ursprünglichen Liste frei wird verwenden.

Insertion Sort

Inplace anstatt mit zweiter Liste

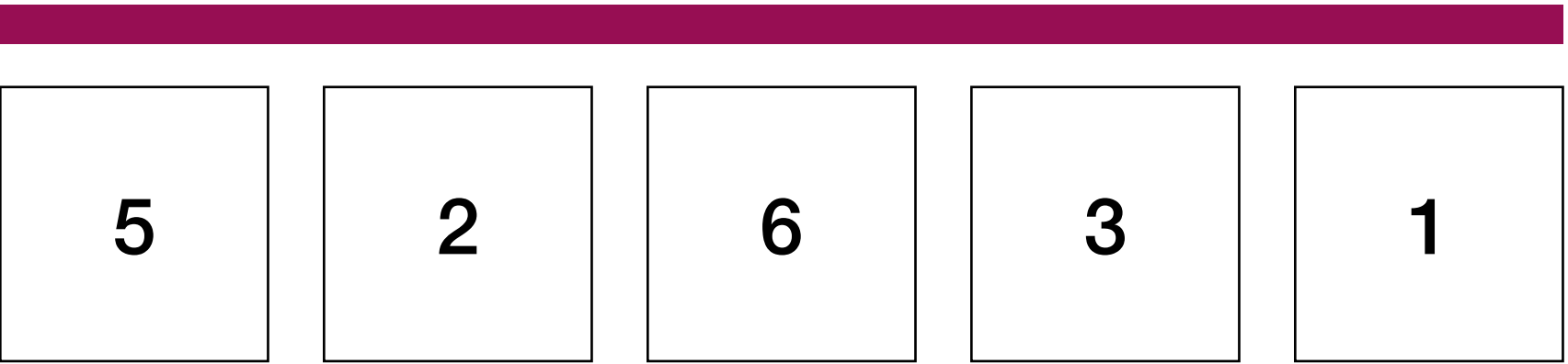
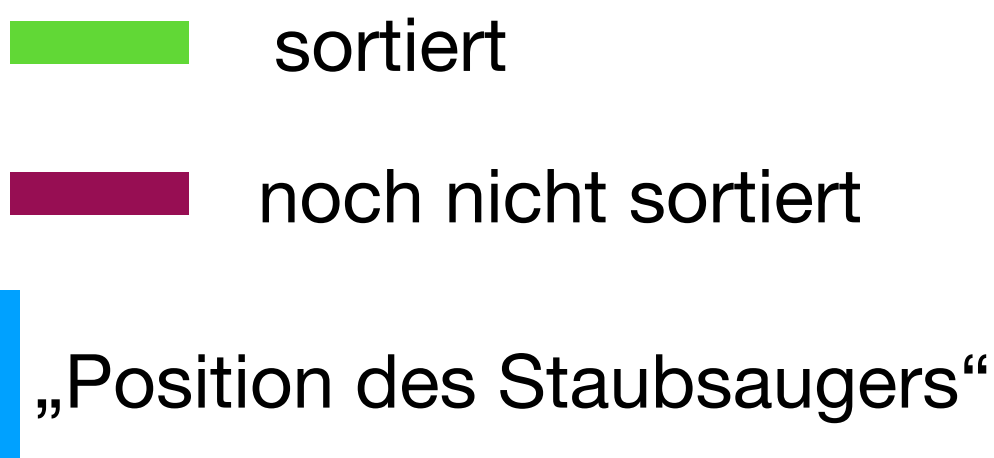


Sortiert

noch nicht sortiert

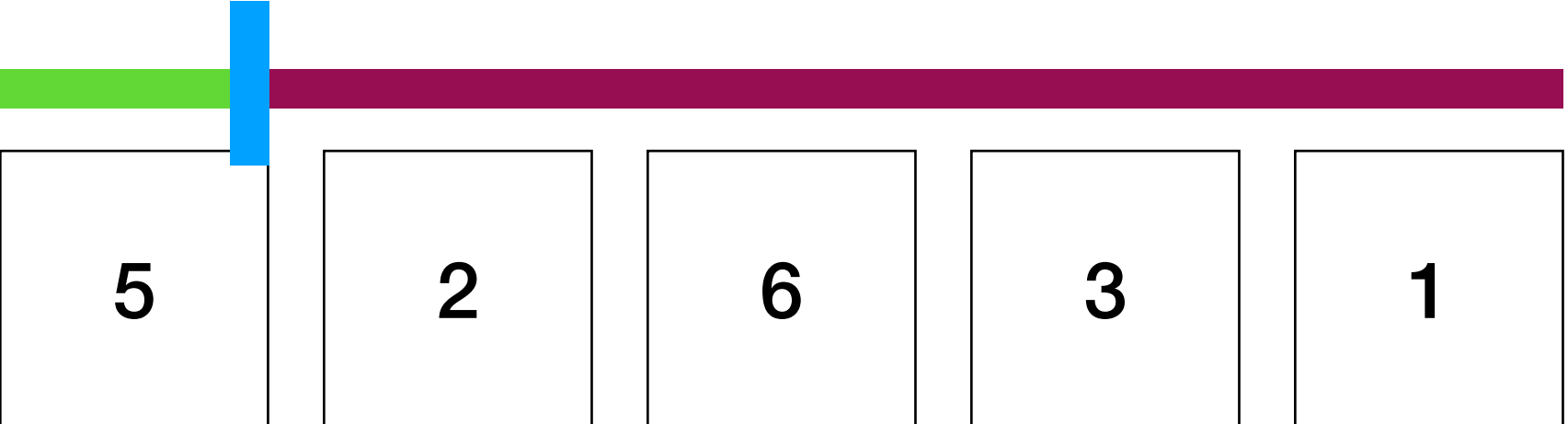


Das Vorgehen

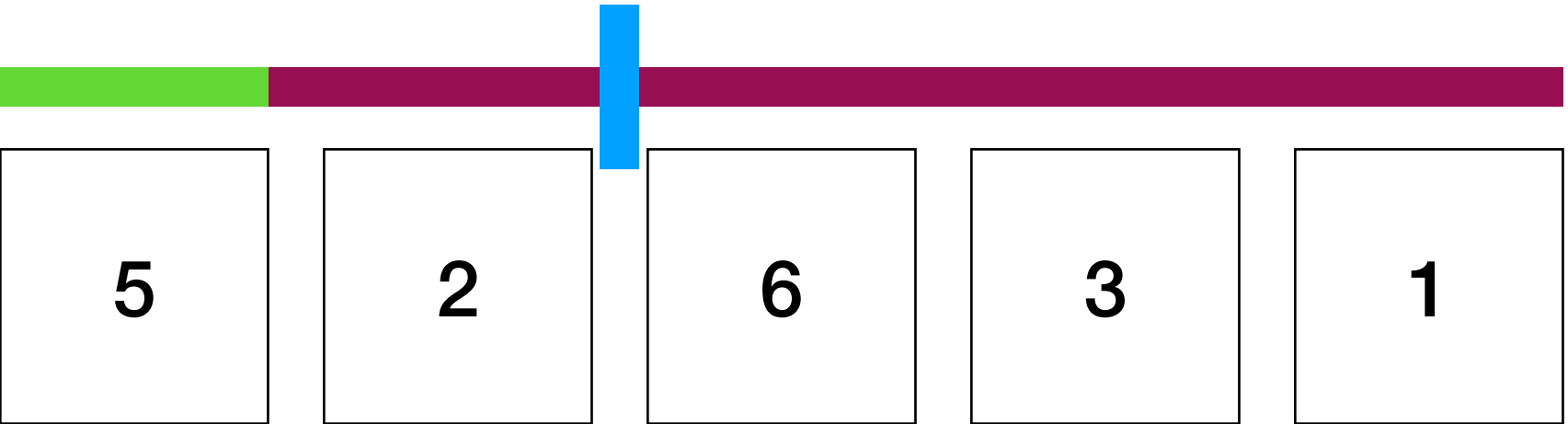


die originale Liste

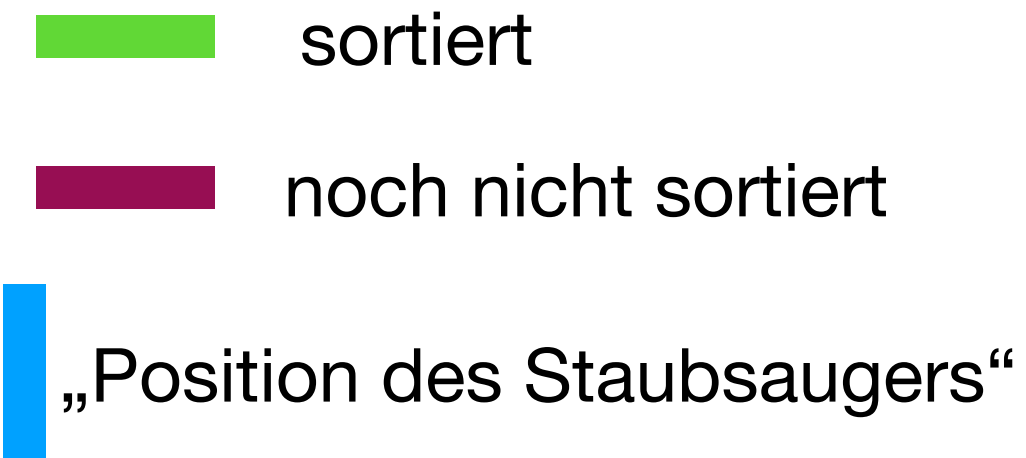
Start Insertion Sort



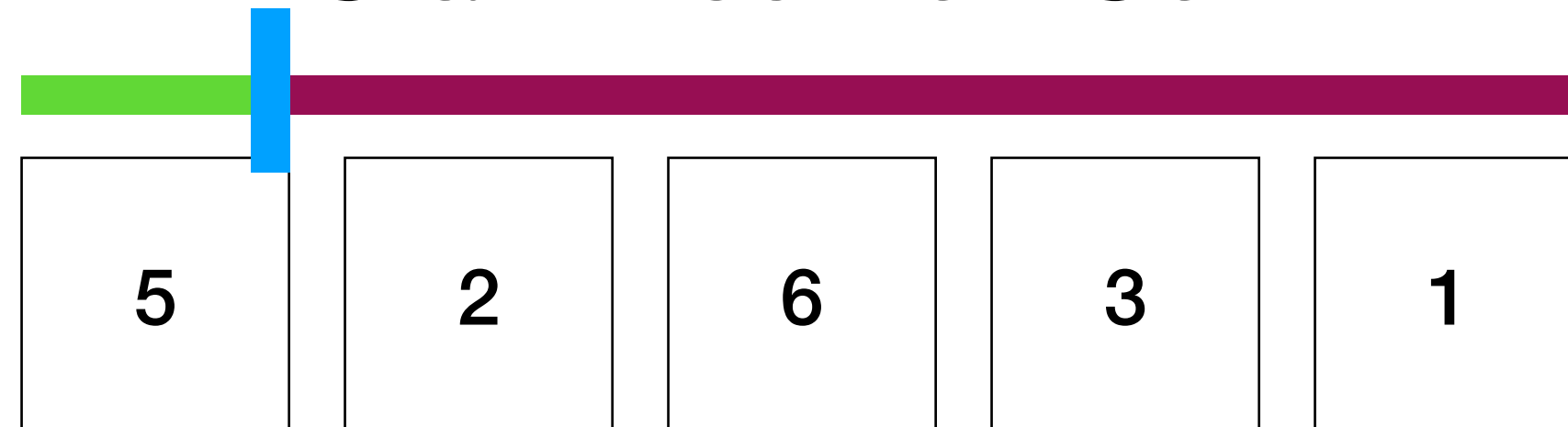
das erste Element ist Automatisch „sortiert“ (es steht innerhalb des grünen Bereichs korrekt)



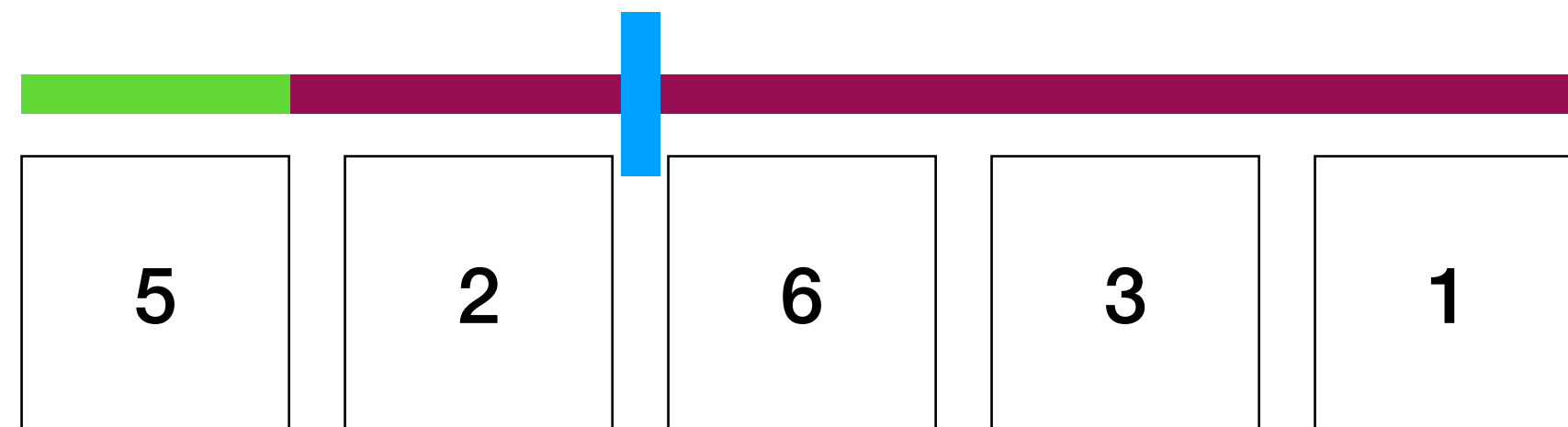
Das Vorgehen



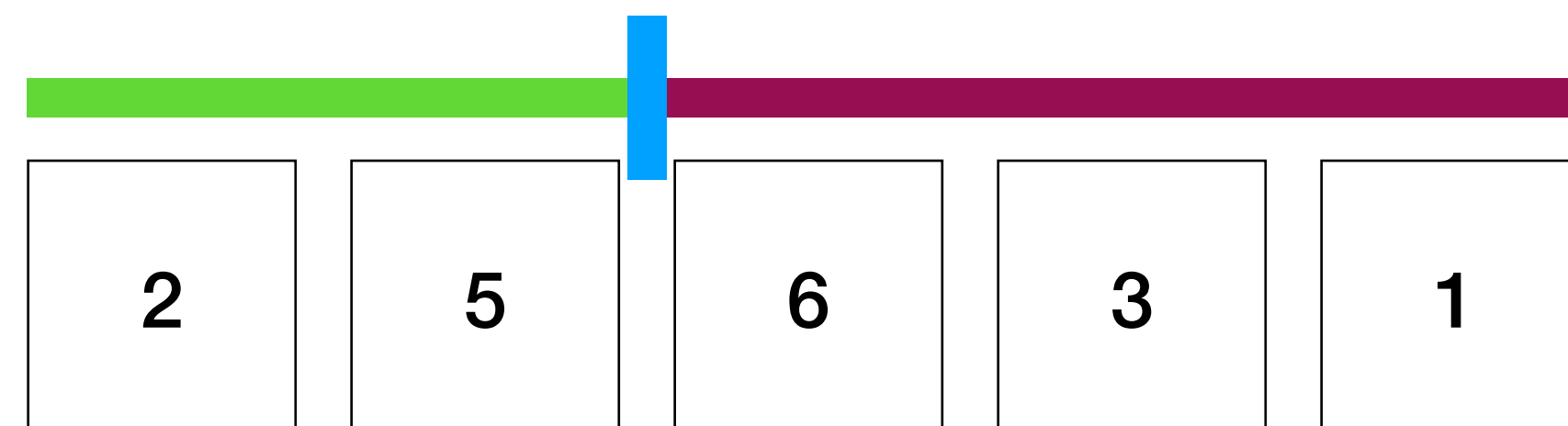
Start Insertion Sort



das erste Element ist Automatisch „sortiert“ (es steht innerhalb des grünen Bereichs korrekt)



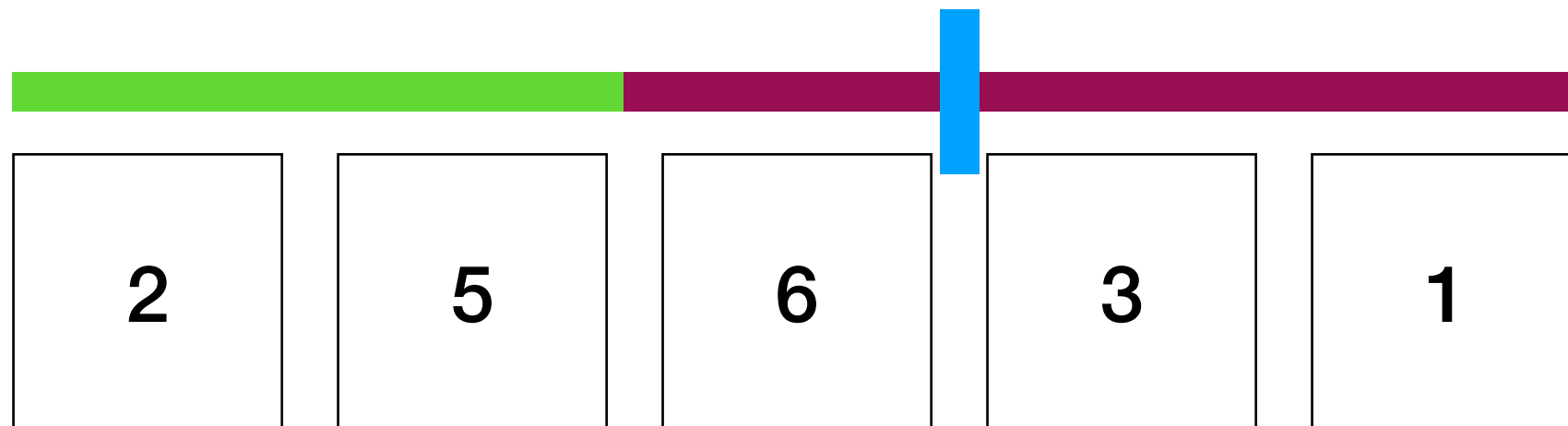
Wir platzieren nun die zwei an der passenden Stelle innerhalb des grünen Bereichs.



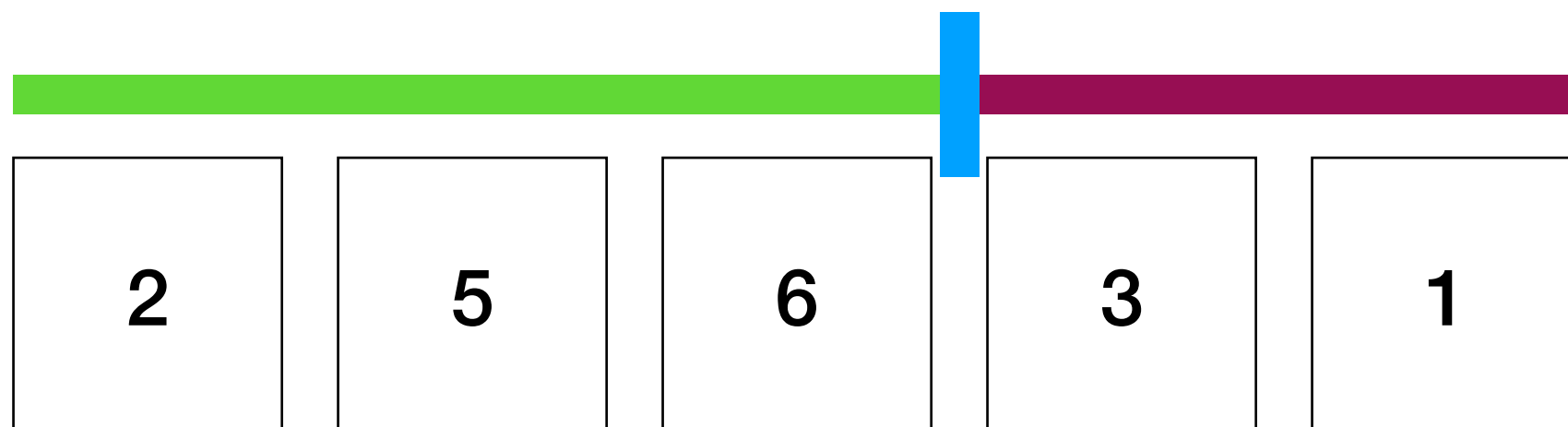
In diesem Fall genügt ein einfaches Tauschen

Das Vorgehen

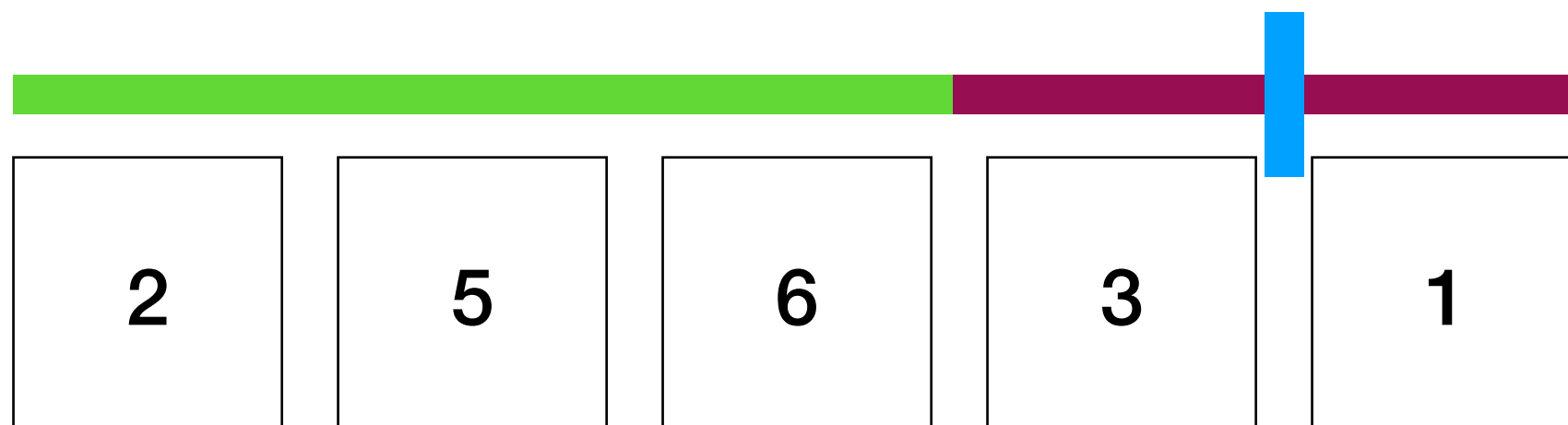
■ sortiert
■ noch nicht sortiert
■ „Position des Staubsaugers“



nun kümmern wir uns um die 6

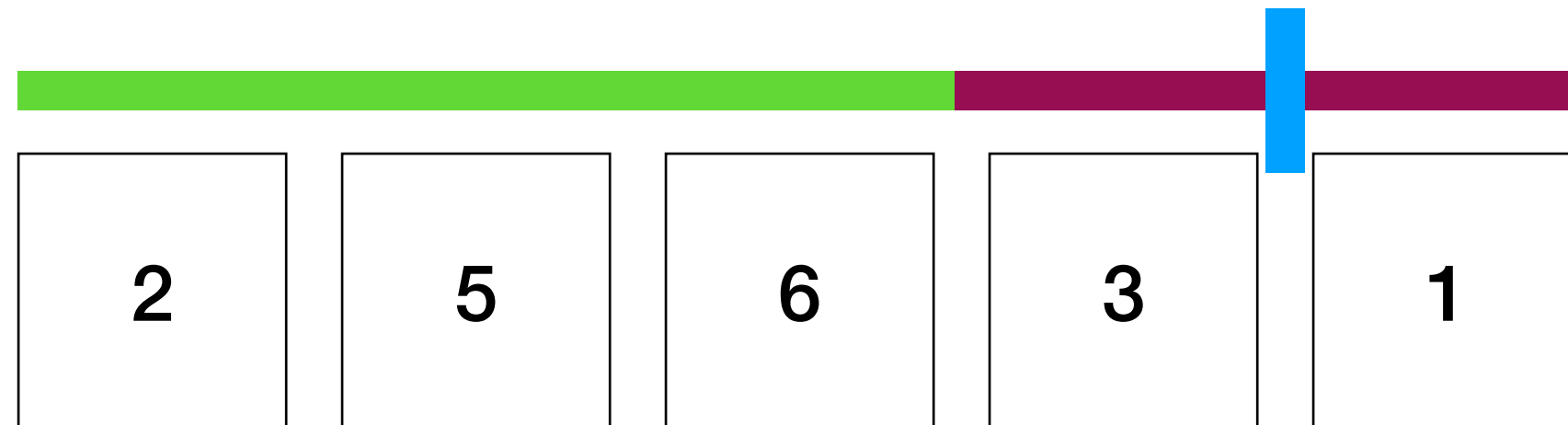
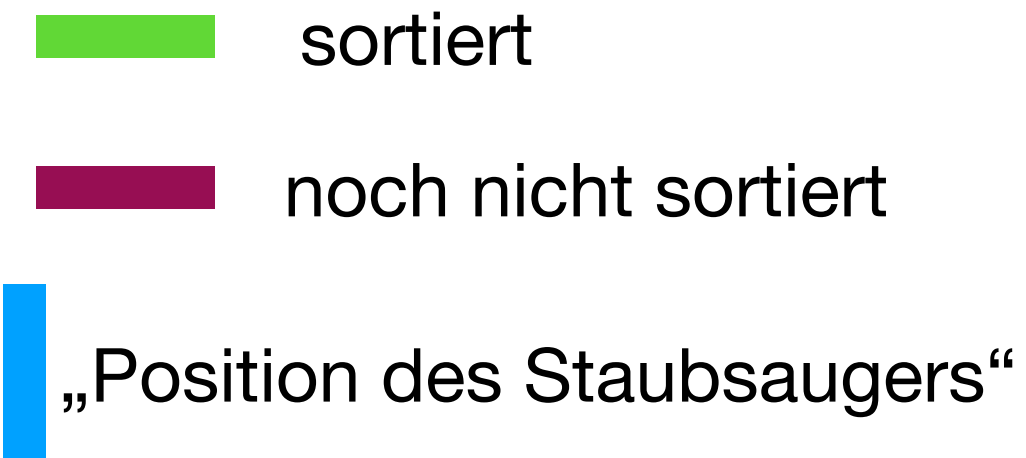


wir merken, dass Sie bereits korrekt steht, also verlängern wir das Grüne einfach.



weiter mit der Drei, nun wird es spannend.

Das Vorgehen

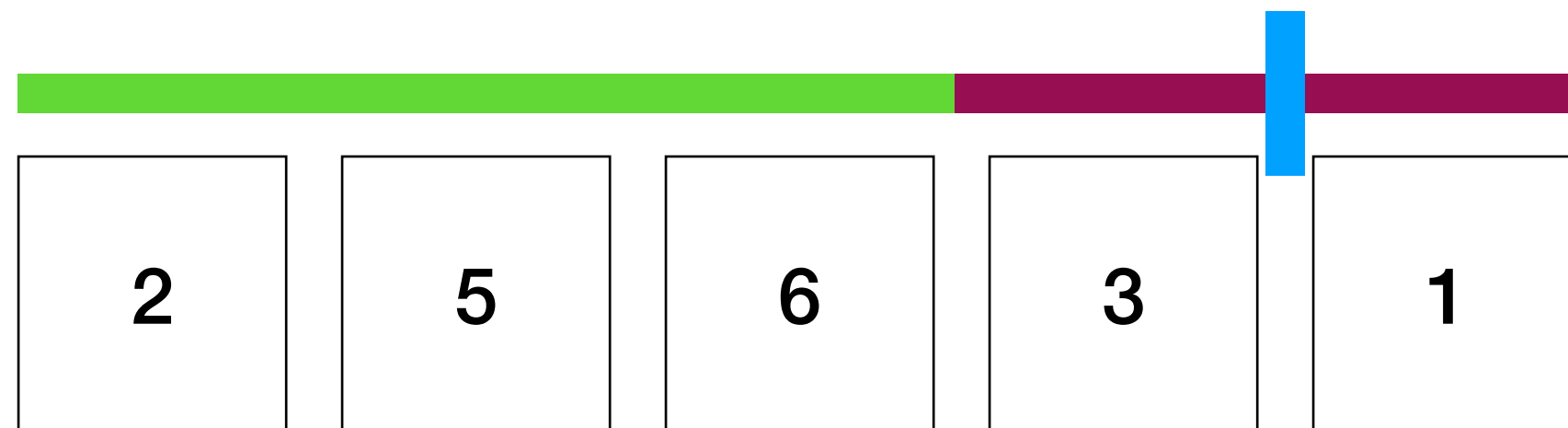
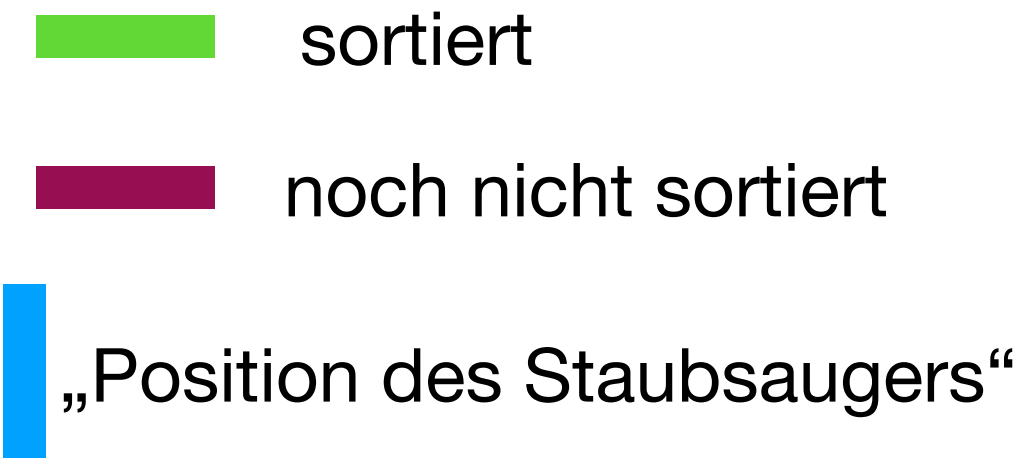


weiter mit der drei, nun wird es spannend.

um die Drei an die korrekte Stelle im Grünenbereich zu bringen, gibt es verschiedene Wege.

Vorschläge? (einfache Lösungen gesucht)

Das Vorgehen



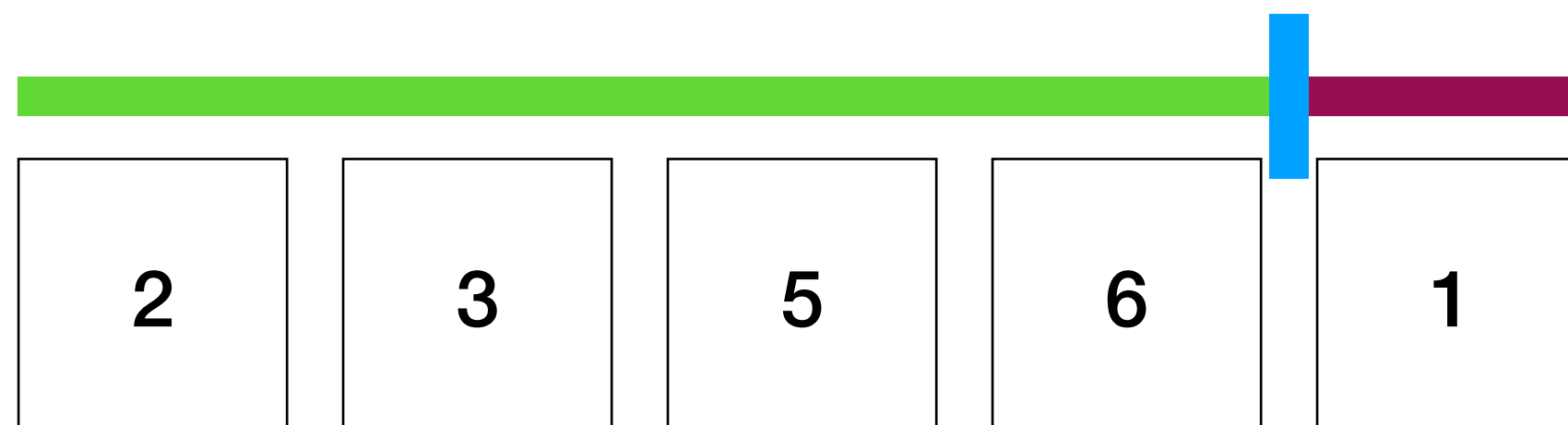
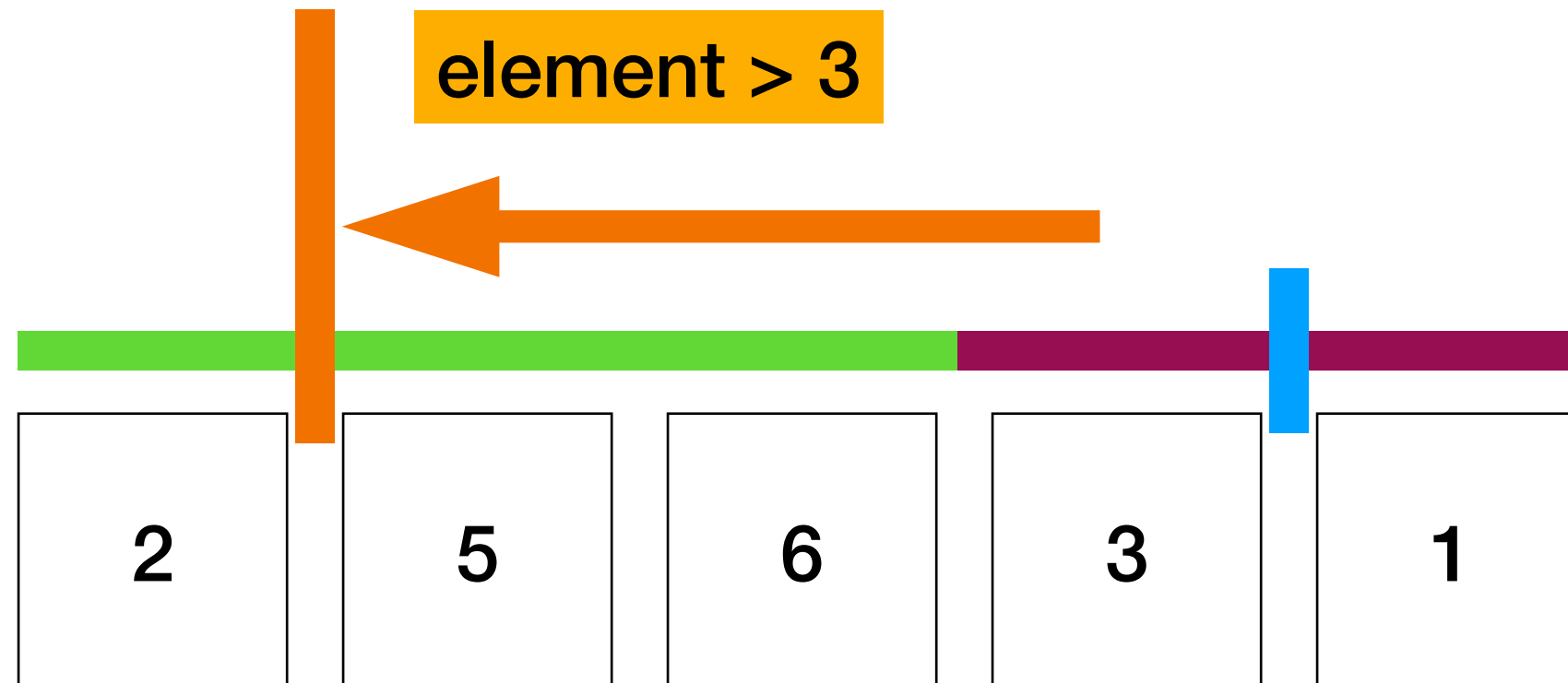
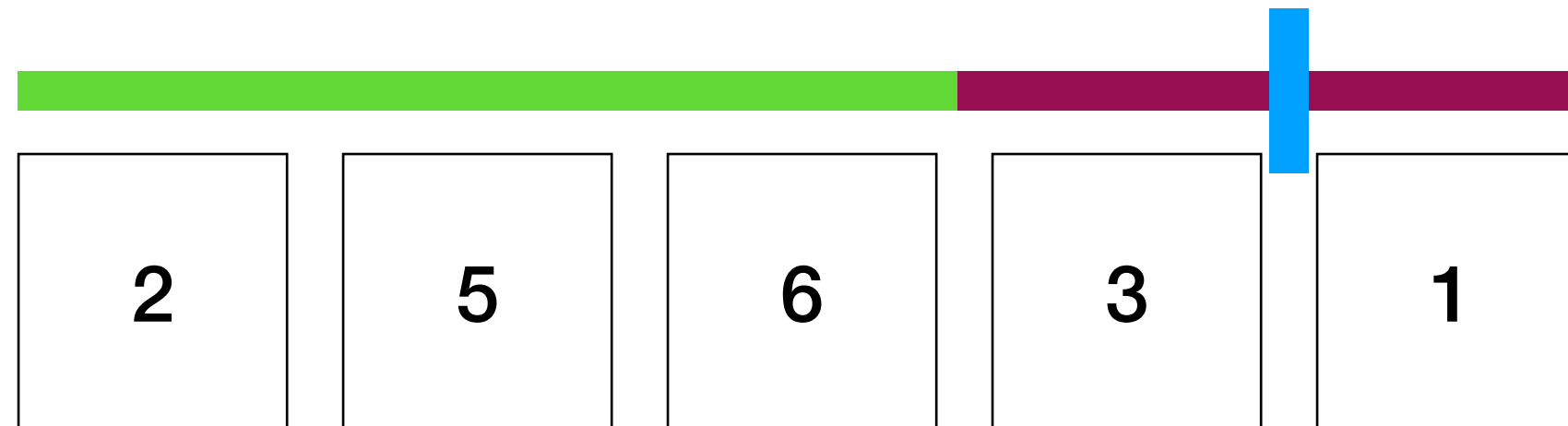
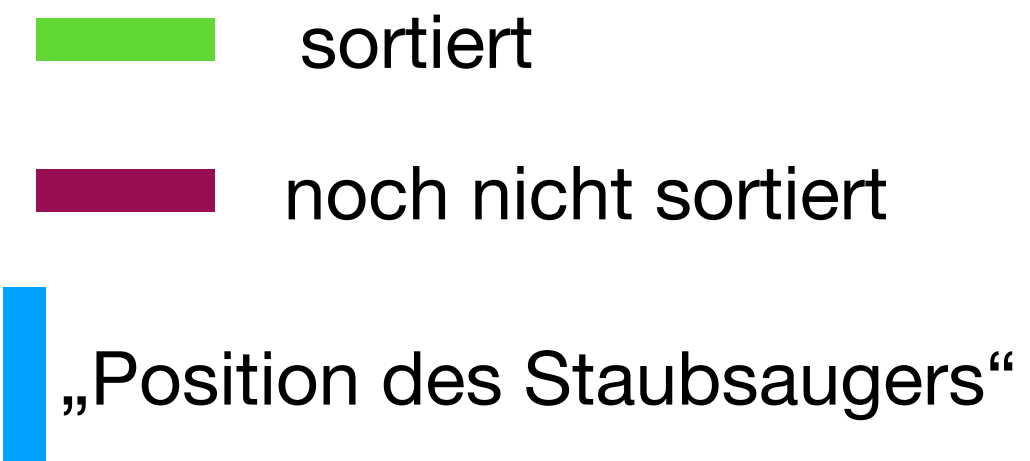
weiter mit der drei, nun wird es spannend.

um die Drei an die korrekte Stelle im Grünenbereich zu bringen, gibt es verschiedene Wege.

Variante 1: Wir iterieren durch den grünen Teil und finden den passenden Index, um die 3 einzusetzen

Variante 2: Wir wenden BubbleSort-„Wandern“ an und swappen die drei bis zur richtigen Position durch. (Standard)

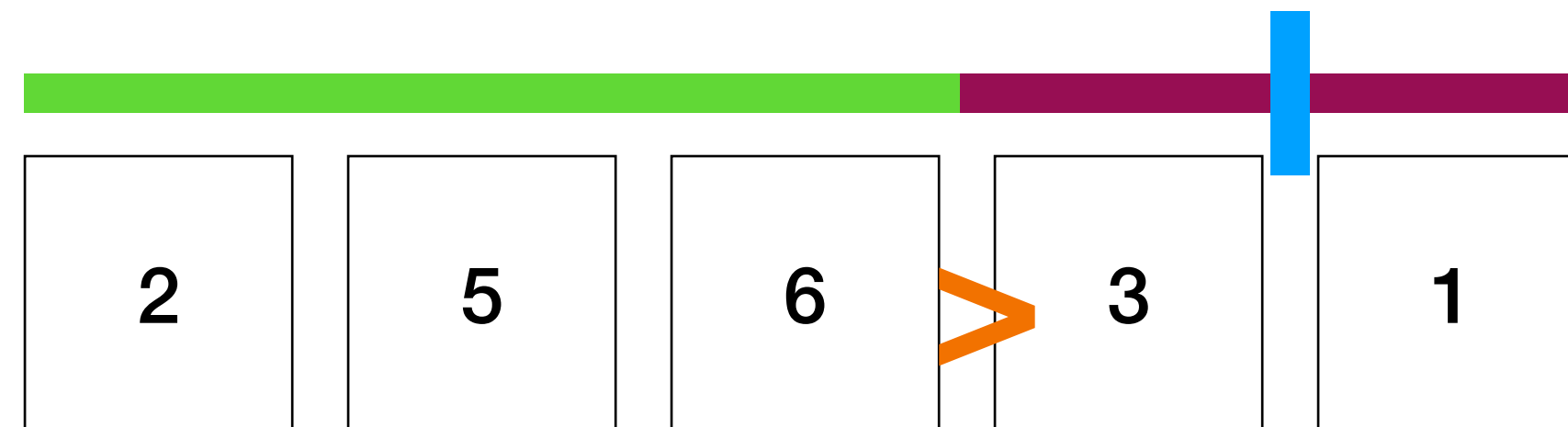
Variante 1: Iterieren um index finden



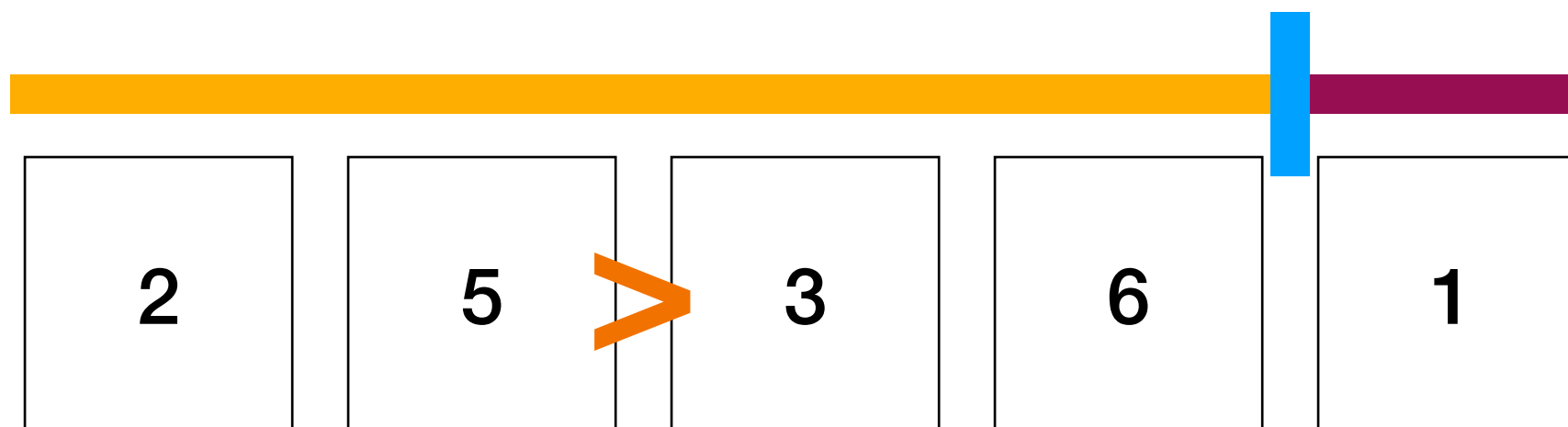
3 eingefügt

Variante 2: BubbleSort-Swappen

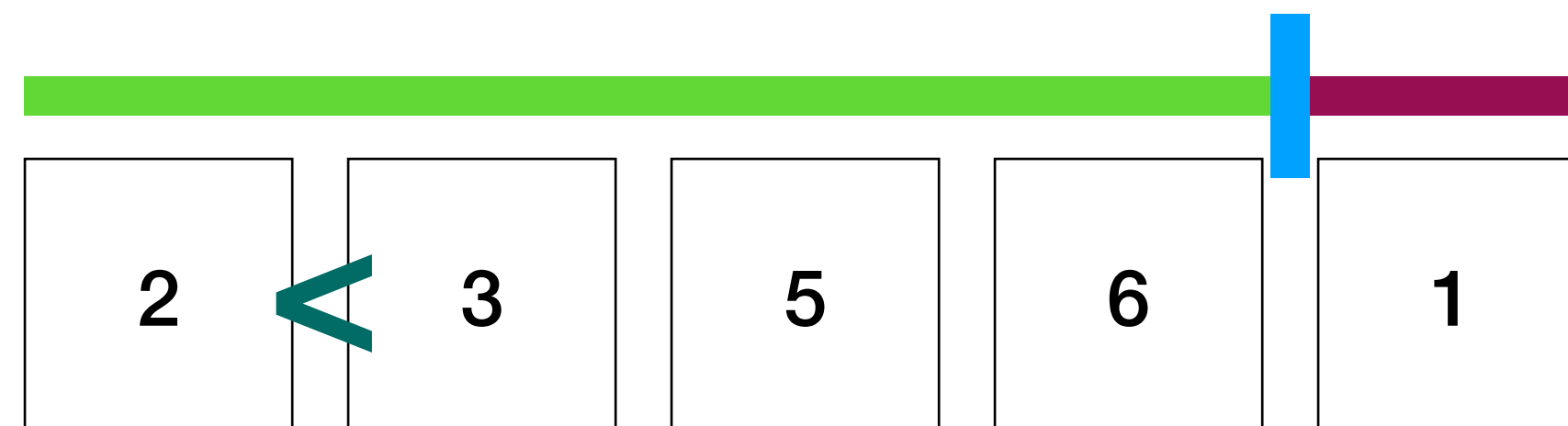
■ sortiert
■ noch nicht sortiert
■ „Position des Staubsaugers“



swappen
mit der eins

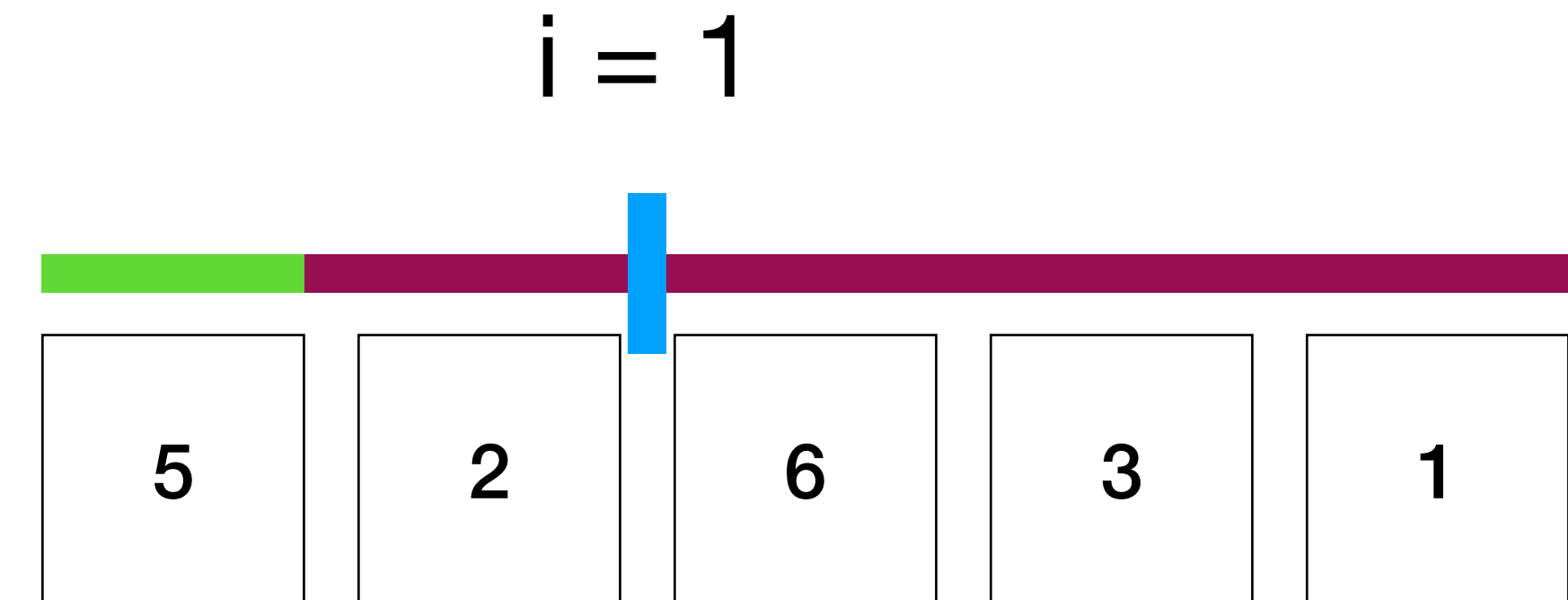


swappen



mit der 1 würde es von hier gleicher massen gemacht werden...

InsertionSort im Code



```
def insertion_sort(items):  
    n = len(items)  
    for i in range(1, n):  
        j = i  
  
        while j > 0 and items[j-1] > items[j]:  
            items[j], items[j-1] = items[j-1], items[j]  
            j -= 1
```

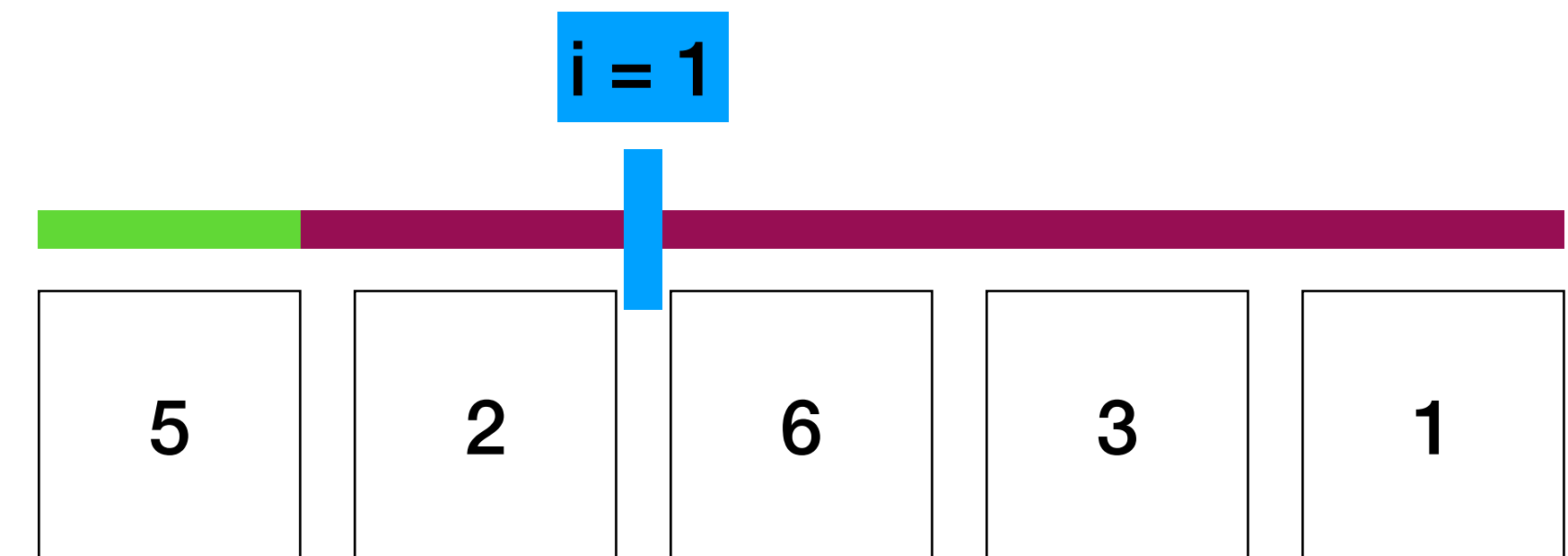
Über die Liste iterieren,
beginnen bei 1 anstatt 0 denn
das erste Element ist ja bereits
sortiert 💡

i ist der „Staubsauger“

InsertionSort im Code

```
def insertion_sort(items):  
    n = len(items)  
    for i in range(1, n):  
        j = i  
        while j > 0 and items[j-1] > items[j]:  
            items[j], items[j-1] = items[j-1], items[j]  
            j -= 1
```

(angenommen i=1)



Wie bei Bubble!

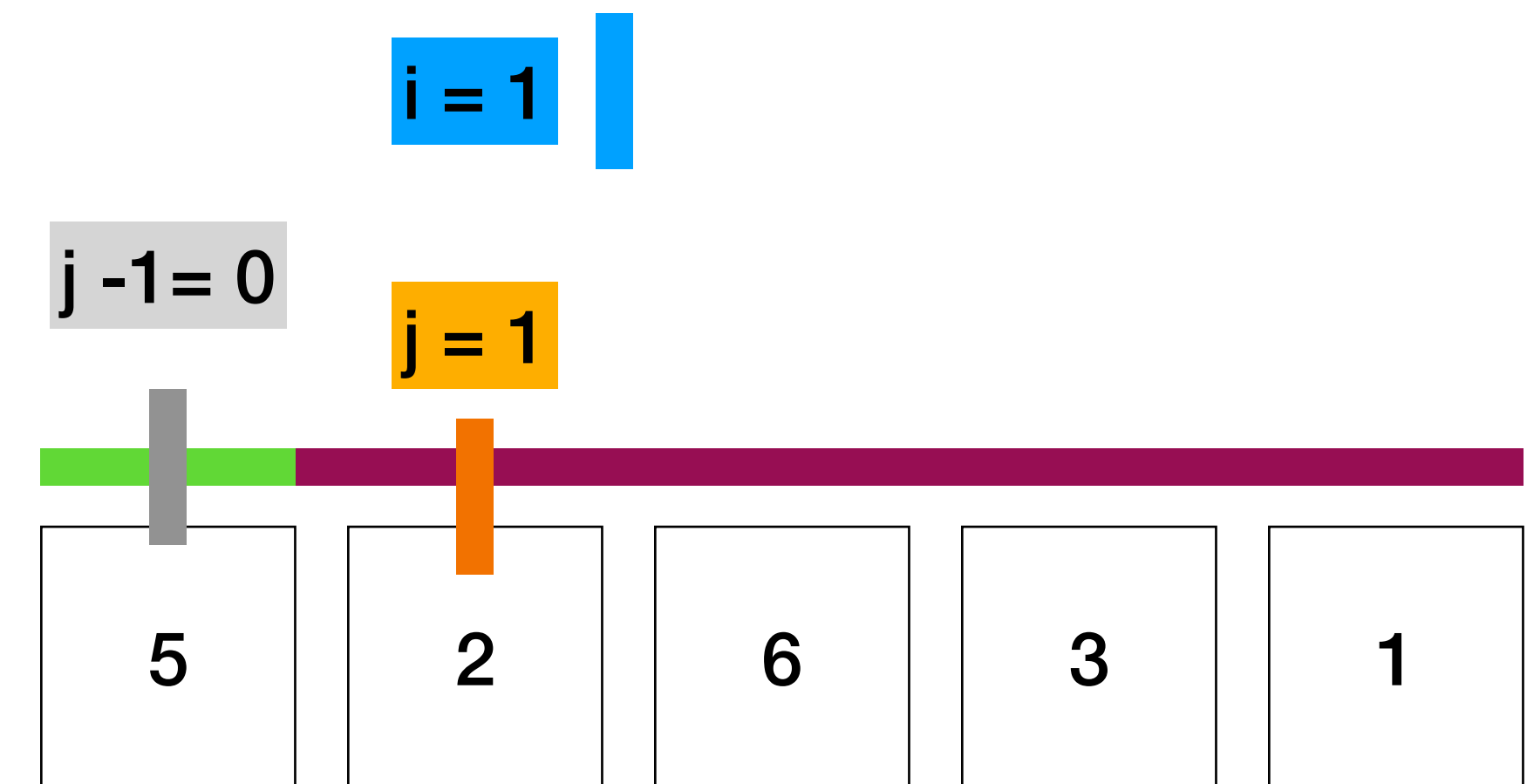
wir tauschen, solange die Bedingungen in while nicht mehr erfüllt sind!

Die Bedingungen sind:

$j > 0$ Index-Sicherheit (trifft zu für ein kleineres Element)

„Das Element links grösser ist, als das einzuordnende Element (2).“

InsertionSort im Code



die **while** schleife:

j=1

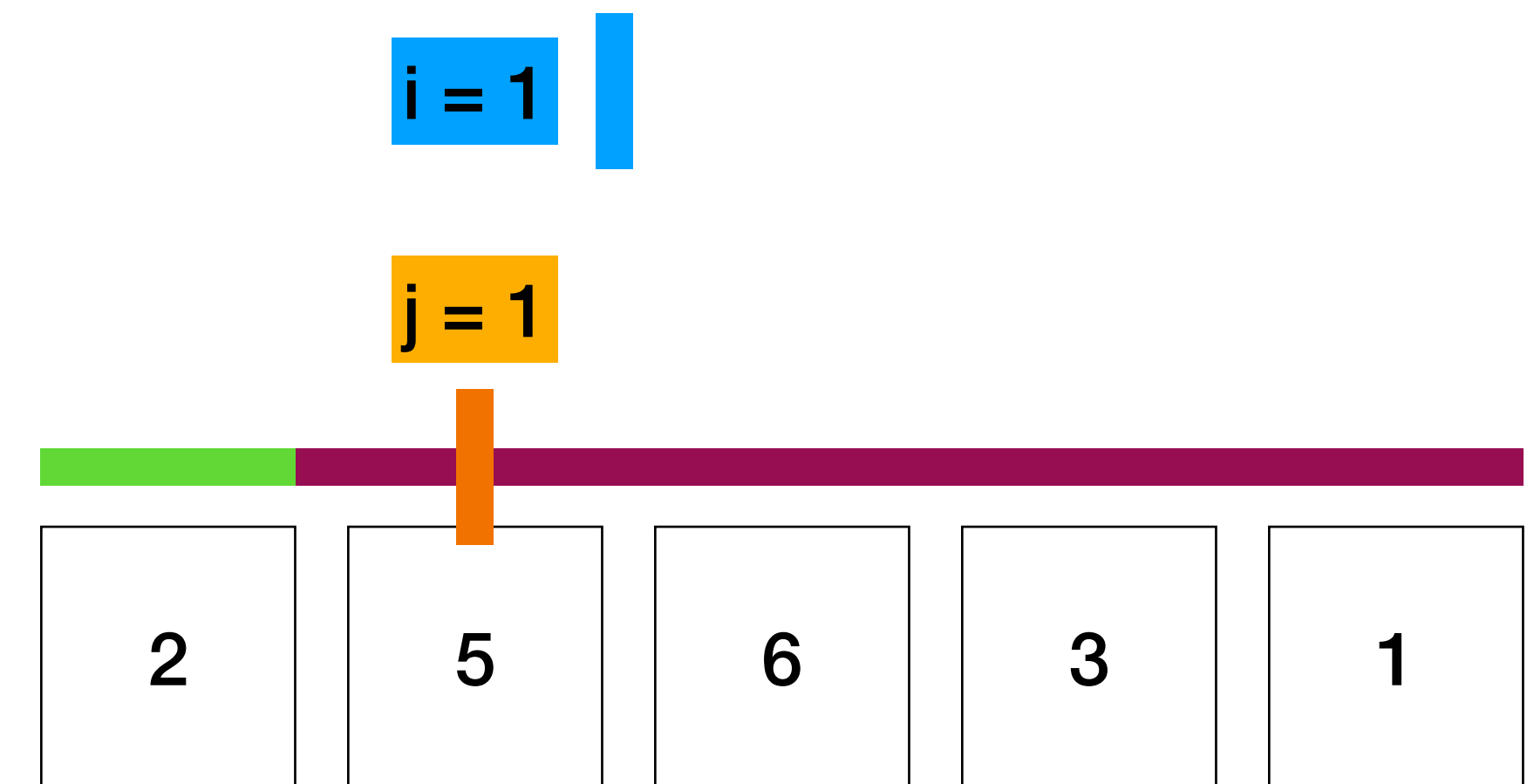
items[1]= **einzuordnendes Element** (2)

items[j-1] = 5

SWAP

```
def insertion_sort(items):  
    n = len(items)  
    for i in range(1, n):  
        j = i                                (angenommen i=1)  
  
        while j > 0 and items[j-1] > items[j]:  
            items[j], items[j-1] = items[j-1], items[j]  
            j -= 1
```

InsertionSort im Code



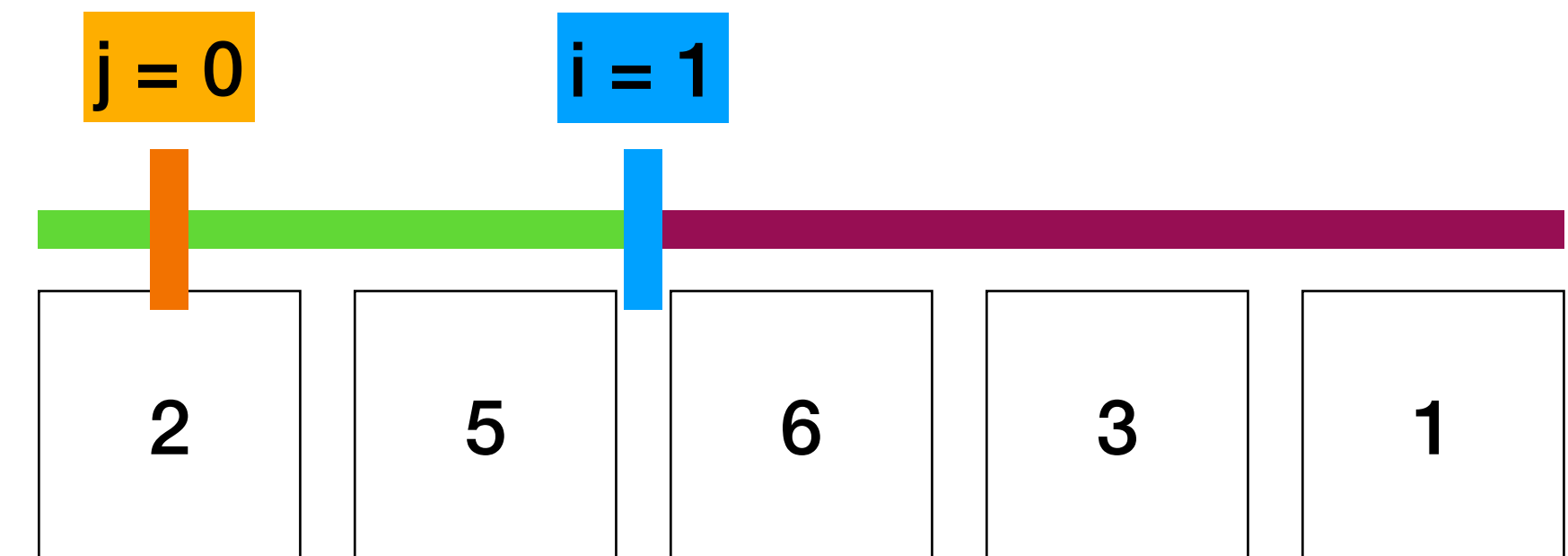
```
def insertion_sort(items):  
    n = len(items)  
    for i in range(1, n):  
        j = i                                (angenommen i=1)  
        while j > 0 and items[j-1] > items[j]:  
            items[j], items[j-1] = items[j-1], items[j]  
            j -= 1
```

Swap durchgeführt.

$j -= 1 \Rightarrow j = 0$

... while schleife $\Rightarrow$ wieder nach oben...

InsertionSort im Code



```
def insertion_sort(items):  
    n = len(items)  
    for i in range(1, n):  
        j = i                                (angenommen i=1)  
        while j > 0 and items[j-1] > items[j]:  
            items[j], items[j-1] = items[j-1], items[j]  
            j -= 1
```

... while schleife=> wieder nach oben...

j ist nicht grösser als 0.

wir sind fertig mit dem Bubble-Wandern.

Insertion 

Laufzeiten

 **Grosser Fokus im theoretischen Teil an der Prüfung** 

Ziel Laufzeiten:

Verhalten einer Funktion einschätzen zu können.

Die Idee: Allgemeine Trends zeigen, bsp.: *Verdoppelt sich die Laufzeit, wenn wir n verdoppeln, oder vervierfacht sie sich?*

Laufzeit verhält sich wie eine Funktion $n \rightarrow n$ oder $n \rightarrow n^2$

Wir **ignorieren** alle konstanten additiven und multiplikativen Faktoren.

Für Funktionen wie $2n$, n und $4n+1$ sprechen wir einfach von einer **Laufzeit von n**

Schranken

Die Laufzeit* unserer Funktion verhält sich....

Oberer Schranke $\mathcal{O}(g)$ **höchstens**, im schlechtesten Fall, **wie g**

Untere Schranke $\Omega(g)$ **mindestens**, im besten Fall, **wie g**

asympt. gleich $\Theta(g)$ **gleich wie g**

...wobei **g** eine andere Funktion/Wachstumsverhalten ist. bsp. $g(n) = n^2$.

* Bei dieser Notation ist es üblich von Laufzeit zu sprechen. Wir interessieren uns aber wirklich nur für das Verhalten, daher kann man bsp. die Anzahl von Funktionsaufrufen als Laufzeit betrachten.

Es handelt sich dabei um Mengen von Funktionen

Intuition:

$f \in \mathcal{O}(g)$: (Laufzeit) f wächst asymptotisch **nicht mehr** als g . Algorithmus mit Laufzeit f ist **nicht schlechter** als einer mit g .

$f \in \Omega(g)$: (Laufzeit) f wächst asymptotisch **nicht weniger** als g . Algorithmus mit Laufzeit f ist **nicht besser** als einer mit g .

$f \in \Theta(g)$: f wächst asymptotisch **gleich schnell** wie g . Algorithmus mit Laufzeit f ist **gleich gut** wie einer mit g .

Recall Anal

$$\log(n) < \sqrt{n} < n < n \cdot \log(n) < n^2 < 2^n < n! < n^n$$

Kurzer Verständnis-Check

Vervollständige Die Lücken

- **Gegeben:** eine Funktion mit oberer Schranke $O(n^2)$.
„Wenn wir n verdoppeln, wird die Laufzeit _____.“
- **Gegeben:** eine Funktion mit unterer Schranke $\Omega(n^2)$.
„Wenn wir n verdoppeln, wird die Laufzeit _____.“

Kurzer Verständnis-Check

- **Gegeben:** eine Funktion mit oberer Schranke $O(n^2)$.
„Wenn wir n verdoppeln, wird die Laufzeit **höchstens vervierfacht**.“
- **Gegeben:** eine Funktion mit unterer Schranke $\Omega(n^2)$.
„Wenn wir n verdoppeln, wird die Laufzeit **mindestens vervierfacht**.“
- Ein Algorithmus hat eine Laufzeit von $T(n)=100n$. Ist die Aussage $T(n) \in O(n^2)$ korrekt?

Kurzer Verständnis-Check

- Ein Algorithmus hat eine Laufzeit von $T(n)=100n$. **Ist die Aussage $T(n) \in O(n^2)$ korrekt? - Ja.** $100n$ wächst nicht schneller als n^2
- Beschreibe das asymptotische Verhalten der Funktion **$f(n) = 5n^2 + 100$.**
- Bestimme die obere Schranke folgender Funktion. $f(n) = 0,001 \cdot 2^n + 10^{10} \cdot n^3$

Kurzer Verständnis-Check

- Beschreibe das asymptotische Verhalten der Funktion **$f(n) = 5n^2 + 100$** .
 $\Theta(n^2)$
- Bestimme die obere Schranke folgender Funktion. $f(n) = 0,001 \cdot 2^n + 10^{10} \cdot n^3$
 $O(2^n)$
- denn $2^n \gg n^3$!

Asymptotische Laufzeiten mit Θ

Snippet 0. (not on CodeExpert)

```
# pre: n is an integer
def run(n):
    for m in range(0, n):
        for k in range(0, n):
            op()
```

- Wie oft wird `op()` aufgerufen?
- Antwort:

Asymptotische Laufzeiten mit Θ

Snippet 1.

```
# pre: n is an integer
def run(n):
    for m in range(0, n):
        for k in range(0, m):
            op()
```

- Wie oft wird `op()` aufgerufen?
- Antwort:

$$\sum_{m=0}^{n-1} \sum_{k=0}^{m-1} 1 = \sum_{m=0}^{n-1} m$$

call = Aufrufen einer Funktion

Summenzeichen verstehen

was passiert hier?

```
def run(n):  
    for m in range(0, n):  
        for k in range(0, m):  
            op()
```

$$\sum_{m=0}^{n-1} \sum_{k=0}^{m-1} 1 = \sum_{m=0}^{n-1} m$$



hier wird einfach nur m-mal 1 summiert => m

Die zweite Schleife ist abhängig von der ersten!

m=0:

k in range(0) => **0** calls. = **#m calls**

m=1:

k in range(1) => **1** call. **#m calls**

m=2:

k in range(2) => **2** calls. **#m calls**

etc.

aufsummieren: **0** + **1** + **2** + etc... bis (n-1)

Asymptotische Laufzeiten mit Θ

Snippet 1.

```
# pre: n is an integer
def run(n):
    for m in range(0, n):
        for k in range(0, m):
            op()
```

- Wie oft wird `op()` aufgerufen?
- Antwort:

$$\begin{aligned} \sum_{m=0}^{n-1} \sum_{k=0}^{m-1} 1 &= \sum_{m=0}^{n-1} m \\ &= \frac{(n-1) \cdot n}{2} \in \Theta(n^2) \end{aligned}$$

Summen und deren Vereinfachungen gehören auf die ZF!

ihr werdet nicht nach derern Herleitung gefragt, aber ihr benötigt die Vereinfachungen um wie hier Θ finden zu können.

Asymptotische Laufzeiten mit Θ

- Wie oft wird `op()` aufgerufen?

Snippet 2.

```
# pre: n is a positive integer
def run(n):
    count = 0
    while n // (2 ** count) >= 1:
        op()
        count += 1
```

Tipp:

$$\log(n) < \sqrt{n} < n < n \cdot \log(n) < n^2 < 2^n < n! < n^n$$

Asymptotische Laufzeiten mit Θ

Snippet 2.

```
# pre: n is a positive integer
def run(n):
    count = 0
    while n // (2 ** count) >= 1:
        op()
        count += 1
```

Auf/Abrundungsoperator

- Wie oft wird **op()** aufgerufen?
- Antwort: $\Theta(\log n)$. Informatik log basis standard 2
- **op()** wird so lange aufgerufen, wie $n/2^{\text{count}} \geq 1$, d.h. solange $\text{count} \leq \log n$
- Beachten Sie, dass *count* die Anzahl der Iterationen in der Schleife verfolgt.
- Daher wird **op()** zwischen $\lfloor \log n \rfloor$ und $\lceil \log n \rceil$ mal aufgerufen.

man kann eine Funktion nicht 3.4-mal aufrufen sondern 3 oder 4 Mal.

Asymptotische Laufzeiten mit Θ

- Wie oft wird `op()` aufgerufen?

Snippet 4.

```
# pre: n is an integer
def run(n):
    for m in range(0, n):
        for k in range(0, m*m):
            op()
```

ZF: AMIV Georg Niggli👑

Tipp: Summen-Formeln aufstellen!

5.1 Häufige Summen

Summe	Vereinfachung	Laufzeit (Θ)
$\sum_{i=1}^n 1$	n	$\Theta(n)$
$\sum_{i=1}^n i$	$\frac{n(n+1)}{2}$	$\Theta(n^2)$
$\sum_{i=1}^n i^2$	$\frac{n(n+1)(2n+1)}{6}$	$\Theta(n^3)$
$\sum_{i=1}^n i^3$	$\left(\frac{n(n+1)}{2}\right)^2$	$\Theta(n^4)$
$\sum_{i=0}^n 2^i$	$2^{n+1} - 1$	$\Theta(2^n)$
$\sum_{i=0}^n a^i \ (a > 1)$	$\frac{a^{n+1} - 1}{a - 1}$	$\Theta(a^n)$
$\sum_{i=1}^n \frac{1}{i}$	$\ln n + \gamma$ (harmonisch)	$\Theta(\log n)$
$\sum_{i=1}^{\log n} i$	$\frac{(\log n)(\log n + 1)}{2}$	$\Theta((\log n)^2)$
$\sum_{i=1}^n \log i$	$\log n! \approx n \log n$	$\Theta(n \log n)$
$\sum_{i=1}^n \sqrt{i}$	$\approx \frac{2}{3} n^{3/2}$	$\Theta(n^{3/2})$
$\sum_{i=1}^n i \log i$	keine einfache Formel	$\Theta(n \log n)$
$\sum_{i=1}^n 2^{n-i}$	$2^n \cdot \sum_{i=1}^n 2^{-i} \approx 2^n$	$\Theta(2^n)$

Asymptotische Laufzeiten mit Θ

Snippet 4.

```
# pre: n is an integer
def run(n):
    for m in range(0, n):
        for k in range(0, m*m):
            op()
```

- Wie oft wird `op()` aufgerufen?
- Antwort:

$$\sum_{m=0}^{n-1} \sum_{k=0}^{m^2-1} 1 = \sum_{m=0}^{n-1} m^2 \\ = \frac{(n-1) \cdot n \cdot (2n-1)}{6}$$

Daraus Folgt.... THETA(?)

Laufzeiten & Sorting

- In welchem **Szenario** erreicht **Insertion Sort** seine schlechteste Laufzeit (Worst Case) von **$O(n^2)$** ?

=> Im Fall einer umgekehrten sortierten Liste müssen n Elemente um $n-i$ Positionen getauscht werden. $n \cdot (n-1) / 2 \Rightarrow n^2$

- Wie hoch ist der zusätzliche Speicherbedarf von BubbleSort? (denkt daran, wie **verhält** sich der Speicherbedarf, bsp bei doppelt so grosser Liste)

BubbleSort operiert In-place, daher $O(1)$ (Grösse der Liste spielt keine Rolle für zusätzlichen Bedarf an Speicher).

- **In welchem Szenario** erreicht **Insertion Sort** seine schlechteste Laufzeit (Worst Case) von **$O(n^2)$** ?
- Wie hoch ist der zusätzliche Speicherbedarf von BubbleSort? (denkt daran, wie **verhält** sich der Speicherbedarf, bsp bei doppelt so grosser Liste)

Exercise 5: Algorithms and Efficiency fällig 30.März

Bemerkungen& Tipps

- **Asymptotische Laufzeit:** Sehr ähnlich wie die kleinen Übungen. Benutzt Schreibzeug!
- **Common Prefix:** Erste Funktion ziemlich simpel. Zweite **rekursiv lösen!** tricky.
- **Dutch Flag:** habe ich als schwer in Erinnerung, aber sehr wichtig konzeptuell!
Befolgt die Invariantenbedingungen!
Reserviert euch etwas Zeit und Geduld für diese Aufgabe.
Nutzt Swaps!
(Schreibt gerne in CX, ob ich nächste Woche darauf eingehen soll)
- **K-Kleinstes Element:** Ebenfalls wichtig und nicht ganz einfach.

Tipps: K kleinstes

Algorithmus, divide and conquer

- Ihr nehmt die Liste euer **Pivot-Element** ist das in der Mitte (+/- in der Mitte) (**Tip**: `Len(list) // 2`)
- **erstellt zwei Listen**, eine (benennt sie z.B links) enthält **alle Elemente kleiner** als das Pivot-Element. die andere (rechts) alle grösseren. (Tipp: list comprehension).
- Vergleicht die Zahl k mit der Länge eurer linken Liste. => Daraus könnt ihr schliessen, ob das gesuchte Element in der Liste links liegt, das Pivot-Element ist oder in der rechten Liste liegt.
- Von hier **rekursiv** weiter verfahren.

Bis nächste Woche :)

...nächste Woche:

- **Rekursive Laufzeitanalyse, wichtig!**
- Divide & conquer

Anonymes Feedback Formular

